

SREGym: 面向 AI SRE 智能体、具备高保真故障场景的实时基准测试

(SREGym: A Live Benchmark for AI SRE Agents with High-Fidelity Failure Scenarios)

Jackson Clark^{◇*} Yiming Su^{◇*} Saad Mohammad Rafid Pial[◇] Yifang Tian[†]
Lily Gniedziejko[◇] Hans-Arno Jacobsen[†] Yinfang Chen[◇] Tianyin Xu[◇]

[◇]University of Illinois Urbana-Champaign (伊利诺伊大学厄巴纳-香槟分校)

[†]University of Toronto (多伦多大学)

* 同等贡献。 arXiv:2605.07161v2 [cs.AI] 13 May 2026

摘要

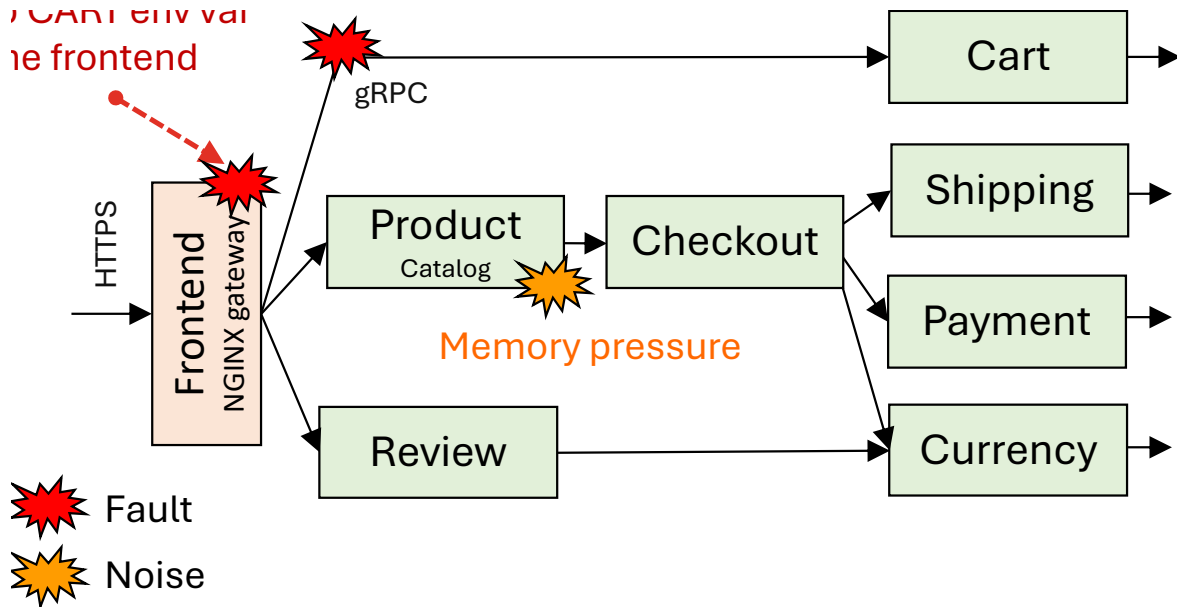
AI 智能体 (agent) 越来越多地被用于诊断与缓解生产系统中的故障, 这一领域被称为智能化的站点可靠性工程 (agentic Site Reliability Engineering, agentic SRE)。当前的 SRE 基准测试仅局限于过度简化的 SRE 任务, 而且由于采用了定制化 (bespoke) 的设计, 遗憾地难以扩展。我们提出 SREGym, 一个面向 SRE 智能体的高保真基准测试。SREGym 暴露出一个构建在真实世界云原生系统栈之上的实时系统环境, 其中通过故障注入器 (fault injector) 来模拟高保真的故障场景。SREGym 通过模拟以下要素来刻画生产环境的复杂性: (1) 系统不同层次上的多种多样的故障 (fault); (2) 各类环境噪声 (ambient noise); 以及 (3) 多样化的失效模式 (failure mode), 例如亚稳态失效 (metastable failure) 和相关性失效 (correlated failure)。SREGym 被架构为一个模块化、可扩展的框架, 它在不同系统栈上编排故障注入器与噪声注入器。SREGym 目前包含 90 个真实、具有挑战性的 SRE 问题。我们使用 SREGym 评测了前沿智能体, 结果表明它们在应对不同种类故障时的能力差异显著, 端到端结果的差距高达 40%。SREGym 作为一个开源项目被持续积极维护, 并已被研究人员和从业者使用。

1 引言

软件生命周期远不止于编写代码, 它还涵盖了对已部署系统在生产环境中的持续运维。尽管编码智能体 (coding agent) 已经变革了软件开发 [14, 20], 但被加速的 (vibe) 编码带来了可靠性挑战: 据报道, AI 生成的代码引入的缺陷比人类编写的代码多 $1.7\times$ [57], 并且 AI 所做代码变更中有 43% 逃过了测试并导致了生产问题 [23]。如今, 一些主要的服务已经在经历由 AI 生成代码所引发的生产宕机 [21]。智能化 AI 的能力必须超越编写代码, 延伸到缓解生产事故, 即站点可靠性工程 (Site Reliability Engineering, SRE)。

SRE 所需的能力总体上不同于编码和软件工程 (Software Engineering, SWE)。SRE 智能体必须跨多模态可观测性数据进行推理 (例如系统配置 [68, 79]、时间序列指标 [49, 65]、非结构化日志 [81, 87] 和分布式追踪 [28, 51, 67])，与领域专用工具交互，并执行多步缓解计划，而这些计划的结果只有在运行时才能观测到。这些要求使 SRE 成为智能体化 AI 中一个独具挑战性的领域。

现有的基准测试不充分，落后于智能体化 SRE 的发展。静态问答 (Q&A) 数据集 [24, 55] 仅局限于领域知识。面向异常检测和根因分析 (root cause analysis, RCA) 的基准测试 [43, 48, 77] 只评测 AI 能否在静态数据集中检测和分析故障，而不评测它们能否解决故障。诸如 AIOpsLab [32] 和 ITBench [50] 等近期工作向前迈进了一步，构建了带有模拟故障的实时系统环境。然而，它们仅局限于过度简化的 SRE 任务。例如，它们主要聚焦于应用层问题。



(a) 某微服务缺失了它向其他服务发送请求所需的环境变量。

Step 1: Scans the system Tool call: <code>kubectl get pods -n astronomy-shop</code> Observation: <i>All pods Running; no crash signal</i>	Step 1: Check cart traces Tool call: <code>get traces('/api/cart')</code> Observation: <i>Cart calls fail while pods healthy</i> ✓
Step 2: Inspect low memory Tool call: <code>kubectl get product-catalog</code> Observation: <i>Suspicious 20MiB memory limit</i> ✗	Step 2: Inspect frontend env. Tool call: <code>kubectl get frontend</code> Observation: <i>CART_ADDR missing</i> ✓
Step 3: Patch product-catalog service Tool call: <code>kubectl set product-catalog</code> Observation: <i>CART_ADDR still unchecked</i> ✗	Step 3: Restore CART_ADDR Tool call: <code>kubectl set env CART_ADDR=:8080</code> Observation: <i>Services recover</i> ✓
...	...
Outcome: Failed Fixates on the low-memory noise; environment variable stays missing	Outcome: Success Restores missing CART_ADDR on frontend service

(b) Stratus (搭配 Sonnet-4.6) 与 Codex (搭配 GPT-5.4) 在处理该问题时的智能体轨迹。

图 1: SREGym 中的一个 SRE 问题, 以及两个智能体在处理该问题时的轨迹。在该失效场景中, 前端 (Frontend) 被丢弃了 CART 环境变量; 前端经 HTTPS/NGINX 网关连接到 Cart、Product Catalog、Checkout、Review、Shipping、Payment、Currency 等微服务 (通过 gRPC), 并叠加了内存压力 (Memory pressure) 噪声。Stratus 固执于低内存噪声, 环境变量始终缺失, 结果失败; Codex 恢复了前端服务上缺失的 CART_ADDR, 结果成功。图例: Fault (故障, 红色星形)、Noise (噪声, 橙色星形)。

然而真实世界的故障在整个系统栈中有着多种多样的根因; 更低层次中的故障, 例如操作系统 [33] 和硬件 [44, 62] 中的故障, 已知更难处理。此外, 它们大多只是把单个故障注入到一个本来干净的环境中, 缺乏在生产环境中并存的噪声和无关事件 [41, 46, 54]。遗憾的是, 由于采用了定制化设计, 这些基准测试难以扩展, 例如把故障模拟硬编码在问题专用脚本中, 且缺乏对分布式事件协调的支持。

本文提出 SREGym，一个面向 SRE 智能体的高保真基准测试。SREGym 与先前工作 [32, 50] 共享同样的高层原则：暴露出一个构建在真实世界云原生系统栈之上的实时系统环境，其中通过故障注入器来模拟故障。¹ 不同之处在于，SREGym 刻画了动态、嘈杂、多事件的生产环境的复杂性，从而实现高保真的故障场景——这些场景不仅能挑战 AI，还能确保问题本身的相关性。SREGym 目前包含 90 个真实、具有挑战性的 SRE 问题。图 1 展示了其中一个问题及相应的智能体行为。SREGym 呈现三项新特性：

1. **模拟系统栈中多种多样的故障**，除了应用层问题之外，还包括硬件故障 [44]、操作系统内核故障 [33, 62]、误操作（misoperation）[38, 69]。
2. **模拟各类环境噪声**——即低影响、与目标故障根因无关的故障，它们引入了歧义并可能造成干扰。
3. **支持多样化的失效模式**，例如亚稳态行为 [29, 45, 47] 以及并发的、相关性的故障 [36, 82]，方法是编排分布式事件（例如故障和噪声）。

SREGym 被架构为一个模块化、可扩展的框架，它把跨系统栈的故障注入器与噪声注入器组合（compose）成高保真的故障场景，作为 SRE 问题。模块化与可扩展性不仅服务于工程纪律以实现可用性与可维护性（这对作为社区驱动基准测试的 SREGym 至关重要；见附录 B），也是其作为一个有用基准测试这一使命的关键促成因素。例如，噪声必须与目标故障一同组合，而亚稳态行为的显现需要对多个相关性故障与事件进行时间上的协调。SREGym 提供了一个统一的编程接口，用于通过对现有故障场景进行变异（例如改变噪声）以及创建新场景来策划（curate）高质量的 SRE 问题。

我们使用 SREGym 评测了一个 SRE 智能体（Stratus [31]）和两个编码智能体（Claude Code [2] 和 OpenAI Codex [5]），分别搭配不同的模型（包括 Sonnet-4.6、GPT-5.4 和 Kimi K2.5）。在各智能体-模型组合中，诊断和缓解的成功率分别为 38.9%–72.6% 和 57.3%–78.5%。这些智能体在处理应用层问题时表现出很强的能力，但在根植于其他层次/模式的故障上成功率显著下降，端到端结果的差距高达 40%。对于复合故障（compound failure），智能体往往得出片面结论，错失全面解决问题的机会。同样地，智能体也以非平凡的方式受到噪声影响。总体而言，通过用高保真故障场景挑战前沿 AI 智能体与模型，SREGym 为推动智能体化 SRE 技术迈向生产就绪奠定了基础。

SREGym 作为一个开源项目在 <https://github.com/SREGym/SREGym> 持续积极维护，已被研究人员和从业者使用。

2 SREGym

构建一个高保真的 SRE 基准测试具有挑战性。该基准测试必须创建一个真实的运维环境，并模拟复杂的故障场景。我们在开发 SREGym 时遵循以下设计原则（附录 B 记录了我们的工程实践）：

¹我们遵循经典的故障-错误-失效（Fault-Error-Failure）模型 [26, 52] 的术语，其中故障（fault）是根因（例如软件缺陷、硬件故障和错误配置）；一个故障可能引发被称为错误（error）的异常行为，错误（若未被妥善处理）会进一步传播并对用户可见，此时被称为失效（failure）。

- **创建嘈杂、多事件的环境。**一个干净、安静的环境不仅不真实，也难以挑战 AI。然而，鲜有现有基准测试对噪声进行建模。SREGym 提供受控的噪声，同时不损害评测的可靠性。
- **模拟故障，而非症状。**我们拒绝现有基准测试中一种常见做法——使用混沌工程(chaos engineering)工具来制造失效症状，这类症状只能通过停止这些工具来缓解。相反，我们聚焦于模拟细粒度的故障。
- **可组合性是扩展问题规模的关键。**SREGym 通过支持把故障与噪声编排成不同的故障场景来实现可组合性 (composability)。先前工作使用 Ansible 脚本，难以扩展，也难以支持需要分布式事件的失效模式。
- **可用性与可扩展性必不可少。**可用性/可扩展性也许并不体现学术新颖性，但对一个基准测试的成功至关重要。SREGym 是一键式 (push-button) 的，并提供用于问题扩展的 API (这使其用户能够贡献新问题)。

图 2 从组件角度概述了 SREGym。我们将在本节其余部分介绍其主要组件。

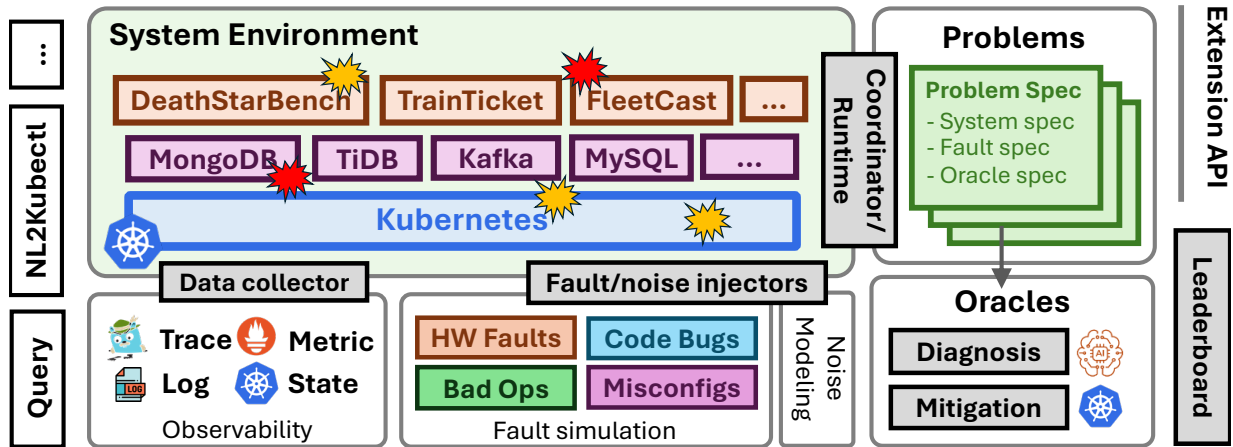


图 2: SREGym 框架与基准测试套件总览。系统环境 (System Environment): 智能体 (Agents, 如 Stratus) 通过工具 (Tools, MCP) 和智能体接口 (Agent Interface) 与之交互。环境包含 DeathStarBench、TrainTicket、FleetCast 等应用, 以及 MongoDB、TiDB、Kafka、MySQL 等后端, 运行在 Kubernetes 之上, 并通过 NL2Kubectrl 操作。数据采集器 (Data collector) 采集 Trace (追踪)、Metric (指标)、Log (日志)、State (状态)。故障/噪声注入器 (Fault/noise injectors) 包含 HW Faults (硬件故障)、Code Bugs (代码缺陷)、Bad Ops (不良操作)、Misconfigs (错误配置) 以及 Noise Modeling (噪声建模)。问题 (Problems) 部分包含协调器/运行时 (Coordinator/Runtime), 问题规约 (Problem Spec) 由系统规约 (System spec)、故障规约 (Fault spec)、判定标准规约 (Oracle spec) 组成; 预言机 (Oracles) 包括诊断 (Diagnosis) 与缓解 (Mitigation)。框架另提供扩展 API (Extension API) 和排行榜 (Leaderboard)。

2.1 问题定义: 用户视角

SREGym 目前包含 90 个 SRE 问题。SREGym 中的一个问题会在一个类生产的实时环境中创建一个失效场景, SRE 智能体必须对其进行诊断与缓解。形式化地, 一个问题是一个四元组 $P =$

(E, I, F, O) , 包括:

- **系统环境 E** 。SREGym 暴露出一个类生产环境, 部署了用户应用、后端系统以及控制/管理服务 (例如 Kubernetes 控制器)。
- **智能体接口 I** 。供智能体观测 E 并与之交互的暴露工具和 API。
- **故障与噪声 F** 。一个集合 $F = \{f_1, \dots, f_k\}$, 包含一个或多个故障, 每个故障针对 E 中某个特定组件。 F 的一个子集可被指定为噪声: 瞬态、低影响的扰动, 智能体必须将其与目标失效的根因区分开来。
- **预言机 (oracle) $O = (O_d, O_m)$** , 其中 O_d 是诊断预言机, O_m 是缓解预言机。

对于每个问题, 受测智能体的输入与期望输出描述如下。

- **输入**。智能体可以通过智能体接口 I , 以标准的可观测性模态来查询系统环境 E (含 F)。
- **期望输出**。智能体对一个问题进行两次提交。第一次是描述故障根因的自然语言诊断。该提交会触发 O_d (见 §2.5)。第二次提交表示智能体已完成其缓解工作, 并基于实际的系统和应用状态触发 O_m 。遵循智能体评测的最佳实践 [22, 92], SREGym 尽可能使用程序化验证以实现可靠的评测。

```
class K8sNetworkPortMisconfig(Problem):
    def __init__(self):
        # Application (应用)
        self.app = SocialNetwork()
        self.app.create_workload()
        # Oracle (预言机)
        self.root_cause = ("The user-service has"
            " a misconfigured network port [...]")
        self.diagnosis_oracle = LLMAJudgeOracle(
            problem=self, expected=self.root_cause)
        self.mitigation_oracle = MitigationOracle(
            problem=self)

    @mark_fault_injected
    def inject_fault(self):
        # Fault (故障)
        injector = NetworkPortFaultInjector(
            namespace=self.namespace)
        injector._inject(
            fault_type="port-misconfig",
            microservice="user-service")
```

图 3: 在 SREGym 中实现一个问题 (噪声由框架注入)。

表 1: SREGym 中的故障与噪声注入器 (“K8s” 指 Kubernetes)。

机制	模拟的故障
杀死一个进程或一个 pod	失停 (fail-stop) 行为
对硬件施压 [12]	慢速失效 (fail-slow) 行为 [42]
通过 eBPF 使系统调用失败	操作系统/硬件故障 [33, 62]
损坏一个磁盘扇区 [19]	磁盘扇区错误 [27, 66]
在 deploy.yaml 中注入故障	服务错误部署
在应用配置中注入故障	应用错误配置 [78]
在 K8s 配置中注入故障	K8s 错误配置 [88]
使用有缺陷的应用代码	代码缺陷
使用有缺陷的应用 operator	误操作 [38, 39]
增加客户端负载	服务过载 [61]
暂停/重启无关的 pod	临时性 pod 失效
注入延迟 / 丢弃数据包	网络延迟 / 抖动
对节点的资源施压	吵闹邻居 (noisy neighbor) [56, 63]

2.2 系统环境

SREGym 暴露出一个云原生、基于 Kubernetes 的系统环境，其中应用部署在 Docker 容器中。一个 SRE 问题会根据失效场景 (§2.4) 来选择部署哪个（哪些）应用。SREGym 自带一个云原生应用目录，包括 DeathStarBench [37]、Train Ticket [91]、Astronomy Shop [15]，以及内部应用（一个卫星轨道模拟器和一个航班预订服务），并搭配后端数据系统（例如 MongoDB、TiDB、Kafka、MySQL 等）。每个应用都有相应的工作负载（例如用户流量）。这些应用和后端系统由 Kubernetes operator 管理。应用、系统和 operator 使用 Helm 包管理器 [6] 部署。该环境构建在事实上的云原生系统栈之上，具有高度可扩展性——它支持任何带有 Kubernetes manifest 或 Helm chart 的真实世界云原生应用和系统，并可部署在任何主流硬件平台上。

2.3 智能体接口

SREGym 对受测智能体的架构或交互模式不作任何假设，以避免智能体与基准测试之间脆弱的耦合。² 该基准测试为智能体暴露出 Model Context Protocol (MCP，模型上下文协议) 服务器，允许它们观测目标系统及其环境并与之交互。SREGym 提供以下接口作为 MCP 服务器：

- **Metrics (指标)**: 通过 Prometheus [10] 查询时间序列性能指标。

²例如，AIOpsLab [32] 把每个失效场景分解为四个相互隔离的子问题（检测、定位、RCA 和缓解），并对它们独立打分；但真实世界的故障处理是一个端到端的单一循环，其中较早的证据和动作会塑造后续的努力；SREGym 则对整个失效场景进行整体性评测。

- **Logs (日志)**: 通过 Loki [8] 搜索和过滤容器日志。
- **Traces (追踪)**: 通过 Jaeger [7] 检查系统组件之间的请求追踪。
- **Cluster control (集群控制)**: 执行任意命令以观测和改变系统状态。我们目前支持 kubectl 命令 (Kubernetes 的命令行接口)。
- **Submission (提交)**: 提交诊断与缓解结果, 从而触发评测。

对于希望设计定制化工具的高级用户, SREGym 也暴露出原始的可观测性端点和 Kubernetes API 端点, 供其智能体连接。

2.4 通过组合故障来创建 SRE 问题

SREGym 提供 90 个问题 (并持续新增)。这些问题可以很容易地被变异, 例如采用不同的噪声模式。SREGym 目前提供 50 个故障原语 (fault primitive), 可应用于 5 个受支持应用中的 139 个可部署服务。在兼容性约束下 (某些故障只能应用于特定服务), SREGym 提供了 3,623 个可行的故障-组件配对, 这还不算组合噪声和多个故障, 相比 90 个场景约有 40× 的倍增 (见附录 C)。这与那些以一组固定硬编码场景形式发布的基准测试有结构性差异: 所策划的 90 个问题反映的是我们已端到端验证过的内容, 而非 SREGym 能够表达的范围。图 3 展示了一个 SREGym 问题的实现。

故障与噪声模拟。表 1 展示了 SREGym 模拟的故障与噪声及其模拟机制。这些故障既包括常见的故障 (如应用错误配置), 也包括 SREGym 新实现的故障 (例如一个使用 eBPF 模拟操作系统和硬件故障的工具)。更多故障注入器可以直接集成到 SREGym 中。这些故障处于系统栈的不同层次, 并通过各种症状显现。它们要求 SRE 智能体对不同系统组件及其交互具有全面的知识与理解。例如, 注入微服务的故障要求 SRE 智能体理解应用逻辑及其依赖关系 [72, 73]。注入 Kubernetes 控制器和 operator 的故障要求 SRE 智能体理解 Kubernetes 如何管理集群和应用 [39, 69]。更底层的操作系统和硬件故障则要求 SRE 智能体理解应用、操作系统和硬件之间的垂直交互 [40]。每个注入器都暴露一个 Python API; 组合一个复合故障可归结为调用若干个注入器。

我们将噪声定义为瞬态、自恢复的扰动 (例如一个 pod 崩溃后被重新调度, 以及少量被丢弃的请求)——它们不是失效的根因。SREGym 以可配置的调度计划注入这些瞬态事件。智能体会看到来自肇事故障和噪声两者的症状, 并且必须区分这两种类型。

失效模式。SREGym 使开发者能够组合故障注入器与噪声注入器, 以创建不同模式下的故障场景。我们当前的问题集涵盖三种重要的失效模式。

- **亚稳态失效 (metastable failure)** 是一种自我维持的拥塞性崩溃: 系统因瞬态事件 (例如负载激增) 而退化, 但在触发因素被移除后仍无法恢复 [29, 45, 47]。亚稳态失效已知难以诊断, 因为它们不以崩溃行为显现。图 4a 展示了我们创建的一个亚稳态失效问题, 它 (1) 为连接超时 (50ms) 和重试次数 (30) 设置了过度激进的 gRPC 配置, (2) 以每秒 3000 个请求的高负载把应用 (DeathStarBench [37] 中的 Hotel Reservation) 推入易损状态, 并 (3) 通过一次瞬态 CPU 压力触发亚稳态失效。如

此一来，所有 RPC 请求同时超时并重试，导致重试风暴（retry storm）。SREGym 协调不同事件的能力对于创建此问题至关重要。

- **并发失效（concurrent failure）**是由多个独立失效同时发生而引起的复合故障。图 4b 描绘了这样一个问题：向应用（DeathStarBench [37] 中的 Social Network）注入了两个故障：（1）一个调度器错误配置，使某个可观测性服务 pod 无法被调度；以及（2）一个网络端口错误配置，使 UserID 服务失效，进而使用户请求失效。该问题评测 SRE 智能体能否理解并优先处理端口错误配置，因为它直接影响服务可用性（调度器错误配置只影响可观测性服务，而该服务对终端用户不可见）。
- **相关性失效（correlated failure）**是指多个组件因共享一个共同原因或依赖而同时失效 [36, 82]。智能体必须识别这些症状之间的相关性，并把它们与底层根因联系起来。

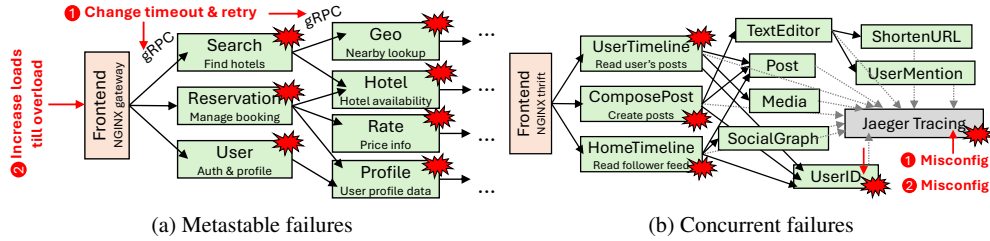


Figure 4: SRE problems that create scenarios with different failure modes in SREGYM

90 scenarios (see Appendix C). This is a structural difference from benchmarks that ship as a fixed set of hardcoded scenarios: the curated 90 problems reflect what we have validated end-to-end, not the extent of what SREGYM can express. Figure 3 shows the implementation of one SREGYM problem.

Fault and Noise Simulation. Table 1 shows the faults and noises SREGYM simulates and the simulation mechanisms. The faults include common ones like application misconfigurations and new

图 4: SREGym 中创建不同失效模式场景的 SRE 问题。(a) 亚稳态失效：在 Hotel Reservation 应用中 ① 更改超时与重试，② 持续增加负载直至过载，通过 gRPC 调用 Search (Find hotels)、Geo (Nearby lookup)、Reservation (Manage booking)、Hotel (Hotel availability)、Rate (Price info)、User、Profile (User profile data)、Auth & profile 等服务。(b) 并发失效：在 Social Network 应用中 ① 错误配置、② 错误配置，涉及 UserTimeline、Post、ComposePost、Media、SocialGraph、HomeTimeline、UserID、TextEditor、ShortenURL、UserMention 等服务以及 Jaeger Tracing 基础设施。

2.5 评测预言机

设计能够公平评测 SRE 智能体的预言机具有挑战性。现有 SRE 基准测试所用的预言机在评测诊断结果时，要求 SRE 智能体严格匹配预定义的标签（例如肇事服务的名称和根因）[32, 75]；这类预言机很脆弱，因为预定义标签往往含糊不清且并非互斥；我们的经验表明，智能体常常报告了正确的结果，却因为脆弱标签不匹配而被错误地评判。ITBench [50] 提出了归一化的拓扑感知匹配（Normalized Topology-Aware Matching），它需要对故障传播图进行细粒度标注。然而，我们发现这种程度的繁重工作容易出错且难以规模化。

诊断预言机。SREGym 采用一种基于清单 (checklist) 的 LLM-as-a-judge 协议 [89], 把诊断评测分解为结构化形式的细粒度问题, 并能在不同评测者之间产生稳定的判定。我们遵循以下原则: 把评测分解为多维度问题以提高评测者间一致性 [53], 并把评分细则锚定在领域专家洞见上, 而非由 LLM 生成的标准 [35, 71]。给定一个真值根因描述 g 和一个智能体提交的诊断 d , 诊断预言机组装一个提示词 (prompt), 其中包含 g 、 d , 以及一份由 $N = 9$ 个是/否问题组成、归入 $K = 3$ 个维度的清单。一个 LLM 评测者针对每个问题 q 返回一个判断 $y_q \in \{0, 1\}$, 并附带支持证据和置信度。对于含问题集 Q_k 的维度 k , 其维度内得分 s_k 为肯定回答所占的比例: $s_k = \frac{1}{|Q_k|} \sum_{q \in Q_k} y_q$; 聚合得分 (aggregated score) 为加权和 $S = \sum_{k=1}^K w_k s_k$ (我们对所有维度赋予相等权重, 即 $w_k = \frac{1}{3}$)。最终判定为 $\hat{v} = \mathbb{I}[S \geq \tau]$, 默认阈值 $\tau = \frac{7}{9}$ (这禁止一份遗漏了整整一个维度的提交通过)。

我们发现, 分解为问题能产生透明的、按维度划分的评测, 揭示智能体的理解在何处出现不足。我们的三个维度是:

- **故障定位 (fault localization):** 诊断是否识别出了正确的起源组件, 并把它与级联失效 (cascading failure) 下游的受害者区分开来?
- **故障刻画 (fault characterization):** 诊断是否捕捉到了故障及其具体细节 (例如错误的端口值、某个特定的环境变量)?
- **失效范围 (failure scope):** 诊断在涉及的组件和症状上是否避免了对影响的过度归因或归因不足?

表 2 在一个分层抽样的 100 个智能体诊断结果上验证了该预言机的判定, 这些结果由一位领域专家独立标注, 并由两个备选 LLM 评测者 (GPT-5.4 和 Kimi K2.5) 评分。默认预言机与人类专家的一致性达到 Cohen’s $\kappa = 0.90$ (几乎完美的一致性), 而两个备选 LLM 评测者收敛于 $\kappa = 0.94$, 并与人类专家保持可观的一致性 (GPT-5.4 和 Kimi K2.5 分别为 $\kappa = 0.70$ 和 $= 0.76$)。LLM 评测者之间的收敛表明, 判定主要取决于清单分解, 而非 LLM 的选择 (这正是单模型 LLM-as-a-judge 评测所担忧的问题 [89])。

表 2: 评测者两两之间的一致性。

评判者 A	评判者 B	一致率	κ
Sonnet-4.6	Human	0.95	0.90
Sonnet-4.6	GPT-5.4	0.88	0.76
Sonnet-4.6	K2.5	0.91	0.82
GPT-5.4	K2.5	0.97	0.94
GPT-5.4	Human	0.85	0.70
K2.5	Human	0.88	0.76

缓解预言机。缓解预言机是问题专用的, 以准确反映目标失效是否真正被缓解。该预言机检查目标故障是否已被解决, 以及目标系统是否已恢复到健康状态。缓解预言机同时使用客户端可观测性 (如

用户请求成功率) 和系统侧可观测性 (应用进程、Kubernetes 集群等)。这使我们能够在复杂的故障场景上评测智能体的缓解结果, 例如亚稳态失效以及底层软件/硬件失效。

3 结果

我们使用 SREGym 评测了三个由前沿 LLM 驱动的 AI 智能体: Stratus [31], 一个最先进的 SRE 智能体, 搭配两个 LLM——Claude Sonnet-4.6 和 Kimi-k2.5; 以及两个编码智能体——搭配 Claude Sonnet-4.6 的 Claude Code [2] 和搭配 GPT-5.4 (gpt-5.4-2026-03-05) 的 Codex [5]。每个智能体-模型组合在每个问题上评测三次运行。我们在整个评测中一致地使用 Claude Sonnet-4.6 作为诊断预言机 (见 §2.5)。

对于噪声模拟, 我们每五分钟随机模拟两种噪声模式, 每种持续两分钟。噪声模拟由框架完成, 并未硬编码在任何问题中。

评测指标。我们以诊断任务和缓解任务的成功率来评测智能体。诊断成功率衡量智能体是否正确定位了目标失效的根因。缓解成功率衡量智能体是否成功缓解了失效 (由缓解预言机验证)。我们还衡量端到端 (end-to-end, E2E) 成功率, 指智能体在同一次运行中既正确诊断又正确缓解的情形。我们还报告诊断耗时 (Time-To-Diagnose, TTD)、缓解耗时 (Time-To-Mitigate, TTM) 以及每次问题运行的平均 token 用量。

表 3: SREGym 上的总体基准测试结果。TTD 和 TTM 被截断在 1800 秒的智能体超时上限, 超时的运行按上限值计入, 因此失败的代价反映在均值中, 而不会导致幸存者偏差 (survivorship bias)。■: 注入噪声的运行; □: 未注入噪声的运行。

智能体	模型	噪声	诊断 (%) ↑	缓解 (%) ↑	E2E (%) ↑	TTD (s) ↓	TTM (s) ↓	# Tokens ↓
Stratus	Sonnet-4.6	□	61.5%	78.5%	54.8%	114.0	771.1	812K
Stratus	Sonnet-4.6	■	51.5%	65.5%	40.2%	170.5	885.0	464K
Stratus	K2.5	□	41.3%	60.6%	32.9%	674.5	1348.8	413K
Stratus	K2.5	■	38.9%	57.3%	30.4%	656.4	1283.2	443K
Claude Code	Sonnet-4.6	□	72.6%	75.6%	60.7%	292.5	702.0	1.47M
Claude Code	Sonnet-4.6	■	62.6%	76.3%	53.7%	314.0	736.5	1.71M
Codex	GPT-5.4	□	70.0%	65.2%	53.3%	176.4	376.0	1.98M
Codex	GPT-5.4	■	59.3%	64.0%	45.9%	218.1	397.7	1.88M

3.1 总体基准测试结果

表 3 展示了总体结果。SREGym 给前沿 AI 智能体与模型带来了挑战: 各智能体-模型组合的诊断成功率为 38.9% 至 72.6%, 缓解成功率为 57.3% 至 78.5%。这些 SRE 问题考察智能体对系统组件之间复杂交互进行推理、从症状推断根因以及有效使用工具来操作系统的能力, 而前沿智能体/模型在

这些方面仍面临困难。

我们发现,智能体在处理 SRE 问题时表现出不同的特性。与 Stratus 和 Codex 相比, Claude Code 表现出最高的端到端成功率。搭配 Sonnet-4.6 的 Stratus 在所有智能体中拥有最高的缓解成功率,这归功于其撤销-重试 (undo-and-retry) 机制 [31]。另一方面,搭配 Kimi K2.5 的 Stratus 成功率最低,原因在于其原始模型能力有限。

Token 成本。 Claude Code 每次运行使用的 token 比 Stratus 多 $3\times$, Codex 多 $3.6\times$ 。原因在于编码智能体并未针对处理 SRE 任务中海量的可观测性数据进行优化。Stratus 作为一个 SRE 智能体,会预处理可观测性数据,仅以相关数据提示 LLM。

噪声的影响。如表 3 所示,每个智能体-模型组合的诊断成功率都有所下降。在噪声下,缓解成功率比诊断更稳健。噪声会干扰智能体关于根因的假设;另一方面,智能体往往能通过自我验证从错误假设中恢复 (见 §3.3)。检查轨迹后我们观察到,所有受测智能体都采取一种贪心 (greedy) 方法 (见附录 D): 它们总是把第一个看似合理的异常当作目标失效 (而这可能是一个噪声)。在若干案例中,智能体确实找到了目标失效根因的证据,却因为它与它们 (错误) 锁定的噪声无关而置之不理。

3.2 新失效场景上的结果

表 4 按失效场景对 SREGym 的问题进行了划分。我们发现,从现有基准测试 [32, 50] 移植而来的问题对搭配强模型的受测智能体带来的挑战有限。例如,搭配 Sonnet-4.6 的 Stratus 的缓解成功率在 80% 以上。这些问题大多是聚焦于应用的单故障场景,且不包含噪声。SREGym 包含一些新问题,它们在故障家族上基于相似的失效场景构建,但在应用和系统组件上有所不同。如表 4 所示,受测智能体表现出相似的结果。

然而,表 4 显示,在 SREGym 独有的新失效场景上,智能体的表现明显更差。搭配 Sonnet-4.6 的 Stratus、Claude Code 和 Codex 的端到端成功率分别从 63.7% 降至 17.9%、从 60.8% 降至 28.2%、从 57.8% 降至 15.4%。这些结果表明, AI 智能体在应对高保真故障 (例如根植于底层栈的故障和/或由复合故障引起的故障) 方面存在显著差距。我们简要描述两个案例研究,更多细节见附录 D。

表 4: 按三种问题类型划分的基准测试结果。“Ported (移植)”指直接从 AIOpsLab/ITBench 移植的问题;“Similar Failures (相似故障)”是 SREGym 中与“Ported”共享失效模式的新问题;“New failures (新故障)”是 SREGym 独有的故障与失效模式。表中 Stratus 的第一、二行对应 Sonnet-4.6, 第三、四行对应 K2.5。

智能体	噪声	Ported ($n = 34$)			Similar Failures ($n = 43$)			New Failures ($n = 13$)		
		诊断	缓解	E2E	诊断	缓解	E2E	诊断	缓解	E2E
Stratus	□	70.6%	83.3%	63.7%	62.8%	80.6%	58.9%	33.3%	59.0%	17.9%
Stratus	■	58.8%	68.6%	45.1%	55.0%	64.3%	44.2%	20.5%	30.8%	10.3%
Stratus	□	42.2%	46.1%	27.5%	46.5%	44.2%	32.6%	15.4%	12.8%	10.3%
Stratus	■	39.2%	49.0%	27.5%	43.1%	42.3%	30.0%	17.9%	23.1%	12.8%
Claude Code	□	74.5%	71.6%	60.8%	81.4%	79.1%	70.5%	38.5%	74.4%	28.2%
Claude Code	■	66.7%	73.5%	52.0%	62.8%	78.3%	56.6%	51.3%	76.9%	48.7%
Codex	□	76.5%	66.7%	57.8%	78.3%	68.2%	61.2%	25.6%	41.0%	15.4%
Codex	■	66.7%	66.7%	51.0%	62.8%	68.2%	50.4%	28.2%	28.2%	17.9%

表 5: 缓解 (M) 以诊断 (D) 结果为条件的条件概率。表中 Stratus 的第一、二行对应 Sonnet-4.6, 第三、四行对应 K2.5。

智能体	噪声	$P(M D)$	$P(M \neg D)$
Stratus	□	0.880	0.588
Stratus	■	0.738	0.506
Stratus	□	0.690	0.351
Stratus	■	0.708	0.396
Claude Code	□	0.798	0.500
Claude Code	■	0.834	0.571
Codex	□	0.734	0.435
Codex	■	0.773	0.431

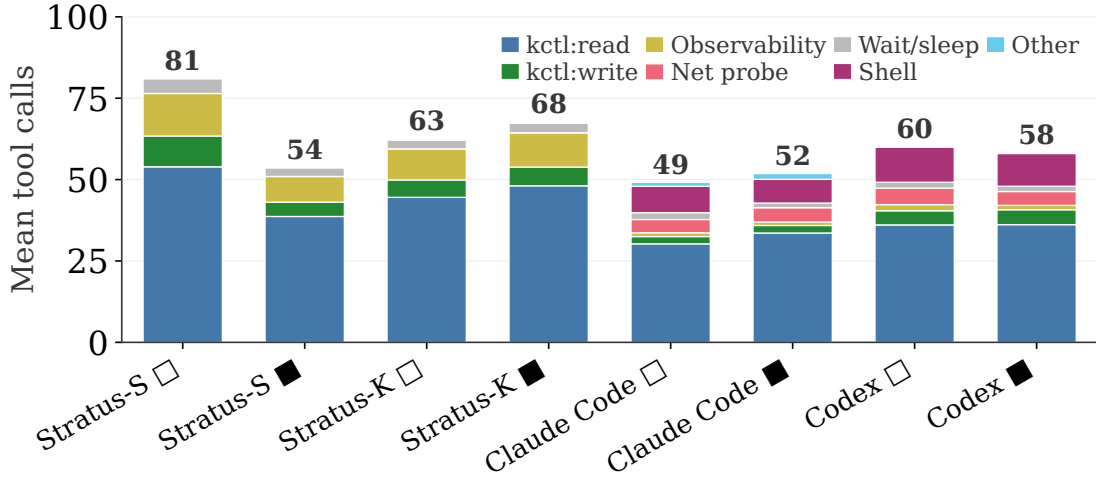


图 5: 按类别统计的每个 SRE 问题的平均工具调用次数。S: Sonnet 4.6, K: Kimi K2.5; ■: 噪声。类别包括: kctl:read (kubectl 读)、kctl:write (kubectl 写)、Observability (可观测性)、Net probe (网络探测)、Shell、Wait/sleep (等待/休眠)、Other (其他)。各配置的平均工具调用次数约为 49–81 次。

硬件故障。 SREGym 的 `latent_sector_error` 问题向一个节点的读系统调用注入间歇性错误。在 Stratus 和 Claude Code 各三次运行中，没有一次运行产生的聚合诊断得分超过 0.22，并且故障刻画在每次运行中都得到 0 分。所有智能体都把这些错误归因于应用内部的错误处理逻辑。没有任何一次运行提出磁盘级诊断（例如读取 `dmesg` 和检查 `smartctl` 输出）。

亚稳态失效。 受测智能体能够可靠地诊断出应用层的触发因素，因为它在分布式追踪和部署配置中可见；然而，在所有亚稳态失效问题中，没有任何一个智能体识别出两个相互作用的组件。在一次运行中，Codex 确实定位到了资源约束，但随后把应用层触发因素当作下游伪影而予以否定，从而产生了一份定位正确但范围错误的诊断 (§2.5)。

总体而言，我们观察到智能体在集群控制/管理平面、已部署系统和用户请求之间缺乏连贯、全面的理解。在亚稳态失效上，智能体没有把系统级约束与应用层触发因素联系起来：而这正是识别系统如何被诱导进入亚稳态所必需的。在硬件失效上，智能体也错失了硬件-软件交互。这种理解的缺乏限制了智能体诊断 SREGym 中复杂故障的能力，而这些故障可以说更接近真实世界的事故。

3.3 诊断与缓解之间的相关性

当一个智能体成功完成缓解时，这一成功是由正确的诊断支撑的，还是智能体在不知所以然的情况下误打误撞地修好了？表 5 展示了缓解以诊断结果为条件的条件概率。智能体可能定位到了根因却在缓解上失败：在智能体正确诊断失效的情形中，缓解仅有 69%–88% 的时间成功 ($P(M | D)$)。当初始诊断未能命中根因时，各智能体的缓解成功率下降 23%–34%。这种下降表明诊断对引导智能体进行缓解是有帮助的。

当诊断错误时，智能体仍可能通过持续观测和调整系统而成功缓解失效。在初始诊断不正确的情况下，各智能体的缓解有 35%–59% 的时间成功 ($P(M | \neg D)$)。注意，该基准测试并不告知智能体其

诊断是否正确。我们观察到两种常见模式：(1) 智能体常常把一个已知症状模式匹配到某些缓解动作上，从而在不理解其根因的情况下成功缓解失效；(2) 智能体能够通过观察失效的持续存在、不断形成新假设并做进一步尝试，来纠正其不正确的诊断。例如，在未注入噪声的情况下，当诊断不正确时，搭配 Sonnet-4.6 的 Stratus 在缓解期间平均进行 3.82 次尝试，而诊断正确时为 1.88 次（见表 12）。

3.4 工具使用

图 5 对智能体轨迹中的工具调用进行了分类（附录 E 展示更多细节）。大多数（60%–72%）工具调用是只读的 `kubectl` 命令。两条命令约占所有读操作的 87%：`kubectl get`（检查系统状态）和 `kubectl logs`（读取容器日志）。被检查的主要资源是 Pod [9]、Deployment [4] 和 ConfigMap [3]。智能体在首次缓解动作之前大约执行 19–28 条读命令（见表 9）。

智能体在缓解动作上有所不同。Stratus 偏好 `kubectl patch`（占写动作的 39%–41%）来改变系统状态，而 Claude Code 和 Codex 偏好 `kubectl rollout`（占写动作的 29%–40%），它会重启 deployment。Codex 还额外依赖 `kubectl run`（19%–21%）来启动临时容器进行集群内网络测试，并依赖 `kubectl port-forward`（11%–12%）进行直接的服务探测，而这些是 Stratus 和 Claude Code 很少使用的工具。

Stratus 拥有定制构建的 API 工具，用于查询指标、追踪和服务依赖图，这些占其工具调用的 15%–17%。Claude Code 和 Codex 则去发现可观测性端点并构造原始的 `curl` 命令；它们的可观测性访问仅占工具调用的 2%–3%。

4 相关工作

AI for SWE（软件工程）的进展推动 AI for SRE 成为下一个前沿。近期工作在智能体化 SRE 技术上取得了积极进展，从根因分析（RCA）到故障缓解 [30, 31, 58, 59, 64, 70, 76, 83, 85, 86, 90]；与此同时，许多商业和开源的 SRE 智能体产品正在被开发 [11, 16–18]。我们与许多 SRE 研究人员和从业者的交流表明，高质量的 SRE 基准测试备受期待。

少数基准测试提供静态数据集，包括问答 [24, 55]、来自真实/合成来源的可观测性数据 [43, 48, 77]，以及系统快照 [75]。SREGym 也可用于生成此类数据集。这些数据集对异常检测和 RCA 很有价值，但存在根本性局限，因为它们不为智能体提供像真实世界诊断过程那样迭代地探测和观测系统的机会。出于同样的原因，这些基准测试也无法支持缓解任务。

SREGym 的设计源自对现有实时基准测试 [32, 50, 84] 的反思——这些基准测试在模拟高保真、真实的故障场景方面能力有限，原因在于缺乏用于编排分布式事件的系统支持、底层故障模拟与注入机制、噪声模拟等。现有实时基准测试还存在一些工程实践问题，例如误用混沌工程工具、缺乏对奖励作弊（reward hacking）的防护等（见附录 B）。

SREGym 聚焦于故障分析与缓解，因此与其他用于部署和回归测试 [75] 以及基于终端的环境搭建 [60] 的相关基准测试是互补的。

5 讨论与结论

SREGym 的愿景是提升 AI for SRE 的标准，并解锁智能体化 SRE 技术全新的演进。怀着这一目标，我们把 SREGym 开发为一个高保真基准测试，用以呈现真实世界生产环境中现有的挑战；同时把它开发为一个可用、可扩展的框架，能够随着面向 AI 的新挑战而持续演进。SREGym 的若干组件还可进一步增强和丰富，包括更多样化的噪声建模、更全面的故障模拟和失效模式，以及系统环境（例如纳入边缘场景）——其中一些工作已经启动。一个明确的下一步是把 SREGym 升级为一个面向 SRE 智能体的强化学习（reinforcement learning, RL）式训练场，超越其当前的问题集。

致谢

我们感谢所有支持、帮助和贡献过 SREGym 的人。具体而言，我们感谢所有 SREGym 的贡献者，以及所有给予我们鼓励和反馈的 SREGym 用户。我们感谢 Braden Hancock、Andy Konwinski、Brighten Godfrey、Sasa Misailovic、Xuan Feng、Lidong Zhou 和 Darko Marinov 对本项目提供的宝贵反馈。SREGym 部分由 Laude Institute 的 Slingshot 资助以及 NSF CNS-2145295 资助支持。

参考文献

1. Chaos mesh: A powerful chaos engineering platform for kubernetes. <https://chaos-mesh.org>.
2. Claude Code by Anthropic. <https://code.claude.com>.
3. ConfigMaps. <https://kubernetes.io/docs/concepts/configuration/configmap>.
4. Deployment. <https://kubernetes.io/docs/concepts/workloads/controllers/deployment>.
5. Codex: Cloud coding agent. <https://chatgpt.com/codex>.
6. Helm - The package manager for Kubernetes. <https://helm.sh>.
7. Jaeger: open source, distributed tracing platform. <https://www.jaegertracing.io>.
8. Loki - a horizontally-scalable, highly-available, multi-tenant log aggregation system inspired by Prometheus. <https://github.com/grafana/loki>.
9. Pods. <https://kubernetes.io/docs/concepts/workloads/pods>.
10. Open source metrics and monitoring for your systems and services. <https://prometheus.io>.
11. TierZero - Agents that handle the incidents, alerts, and internal questions that fragment your team's day. <https://www.tierzero.ai/>.
12. stress-ng - a tool to load and stress a computer system. <https://wiki.ubuntu.com/Kernel/Reference/stress-ng>, 2013.

13. Chaosblade: An Easy to Use and Powerful Chaos Engineering Toolkit. <https://github.com/chaosblade-io/chaosblade>, 2019.
14. Quantifying GitHub Copilot’s Impact in the Enterprise with Accenture. <https://github.blog/news-insights/research/research-quantifying-github-copilots-impact-in-the-enterprise-with-a> May 2024.
15. Otel-Demo - A microservice-based distributed system intended to illustrate the implementation of OpenTelemetry in a near real-world environment. <https://github.com/open-telemetry/opentelemetry-demo>, 2024.
16. AWS DevOps Agent. <https://aws.amazon.com/devops-agent/>, 2025.
17. Azure SRE Agent. <https://azure.microsoft.com/en-us/products/sre-agent>, 2025.
18. Ciroos - Reduce toil, investigate incidents faster, and drive autonomous operations. <https://ciroos.ai/>, 2025.
19. dm-dust - A Linux kernel module which can be used to simulate the bad blocks behavior on a physical disk. <https://docs.kernel.org/admin-guide/device-mapper/dm-dust.html>, 2025.
20. 2025 Stack Overflow Developer Survey. <https://survey.stackoverflow.co/2025/>, 2025.
21. Amazon tightens code controls after outages, including one caused by AI. <https://www.businessinsider.com/amazon-tightens-code-controls-after-outages-including-one-ai-2026-3>, Mar. 2026.
22. Demystifying Evals for AI Agents. <https://www.anthropic.com/engineering/demystifying-evals-for-ai> 2026.
23. 43% of AI-generated code changes need debugging in production, survey finds. <https://venturebeat.com/technology/43-of-ai-generated-code-changes-need-debugging-in-production-survey-finds>, 2026.
24. SRE-skills-bench: LLM Benchmark for SRE Tasks. <https://github.com/Rootly-AI-Labs/SRE-skills-bench>, 2026.
25. A. Alquraan, H. Takruri, M. Alfatafta, and S. Al-Kiswany. An Analysis of Network-Partitioning Failures in Cloud Systems. In Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI’18), Oct. 2018.
26. A. Avizienis, J.-C. Laprie, B. Randell, and C. Landwehr. Basic Concepts and Taxonomy of Dependable and Secure Computing. IEEE Transactions on Dependable and Secure Computing (TDSC), 1(1):1–23, Jan. 2004.
27. L. N. Bairavasundaram, G. R. Goodson, S. Pasupathy, and J. Schindler. An Analysis of Latent Sector Errors in Disk Drives. In Proceedings of the 2007 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems (SIGMETRICS’07), June 2007.

28. P. Barham, A. Donnelly, R. Isaacs, and R. Mortier. Using Magpie for Request Extraction and Workload Modelling. In Proceedings of the 6th USENIX Symposium on Operating Systems Design and Implementation (OSDI'04), Dec. 2004.
29. N. Bronson, A. Aghayev, A. Charapko, and T. Zhu. Metastable Failures in Distributed Systems. In Proceedings of the 18th Workshop on Hot Topics in Operating Systems (HotOS'21), June 2021.
30. Y. Chen, H. Xie, M. Ma, Y. Kang, X. Gao, L. Shi, Y. Cao, X. Gao, H. Fan, M. Wen, J. Zeng, S. Ghosh, X. Zhang, C. Zhang, Q. Lin, S. Rajmohan, D. Zhang, and T. Xu. Automatic Root Cause Analysis via Large Language Models for Cloud Incidents. In Proceedings of the 19th European Conference on Computer Systems (EuroSys'24), Apr. 2024.
31. Y. Chen, J. Pan, J. Clark, Y. Su, N. Zheutlin, B. Bhavya, R. Arora, Y. Deng, S. Jha, and T. Xu. Stratus: A Multi-agent System for Autonomous Reliability Engineering of Modern Clouds. In Proceedings of The 39th Annual Conference on Neural Information Processing Systems (NeurIPS'25), Dec. 2025.
32. Y. Chen, M. Shetty, G. Somashekar, M. Ma, Y. Simmhan, J. Mace, C. Bansal, R. Wang, and S. Rajmohan. AIOpsLab: A Holistic Framework to Evaluate AI Agents for Enabling Autonomous Clouds. In Proceedings of the 8th Conference on Machine Learning and Systems (MLSys'25), May 2025.
33. A. Chou, J. Yang, B. Chelf, S. Hallem, and D. Engler. An Empirical Study of Operating Systems Errors. In Proceedings of the 18th ACM Symposium on Operating Systems Principles (SOSP'01), Oct. 2001.
34. M. Chow, Y. Wang, W. Wang, A. Hailu, R. Bopardikar, B. Zhang, J. Qu, D. Meisner, S. Sonawane, Y. Zhang, R. Paim, M. Ward, I. Huang, M. McNally, D. Hodges, Z. Farkas, C. Gocmen, E. Huang, and C. Tang. ServiceLab: Preventing Tiny Performance Regressions at Hyperscale through Pre-Production Testing. In Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI'24), July 2024.
35. S. Y. Chu, J. W. Kim, and M. Y. Yi. Think Together and Work Better: Combining Humans' and LLMs' Think-Aloud Outcomes for Effective Text Evaluation. In Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI'25), Apr. 2025.
36. D. Ford, F. Labelle, F. I. Popovici, M. Stokely, V.-A. Truong, L. Barroso, C. Grimes, and S. Quinlan. Availability in Globally Distributed Storage Systems. In Proceedings of the 9th USENIX Symposium on Operating Systems Design and Implementation (OSDI'10), Oct. 2010.
37. Y. Gan, Y. Zhang, D. Cheng, A. Shetty, P. Rathi, N. Katarki, A. Bruno, J. Hu, B. Ritchken, B. Jackson, K. Hu, M. Pancholi, Y. He, B. Clancy, C. Colen, F. Wen, C. Leung, S. Wang, L. Zaruvinsky, M. Espinosa, R. Lin, Z. Liu, J. Padilla, and C. Delimitrou. An Open-Source

- Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud & Edge Systems. In Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'19), Apr. 2019.
38. J. T. Gu, X. Sun, W. Zhang, Y. Jiang, C. Wang, M. Vaziri, O. Legunsen, and T. Xu. Acto: Automatic End-to-End Testing for Operation Correctness of Cloud System Management. In Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP'23), Oct. 2023.
39. J. T. Gu, Z. Tang, Y. Su, B. A. Stoica, X. Sun, W. X. Zheng, Y. Zhang, A. Rahman, C. Wang, and T. Xu. Who Watches the Watchers? On the Reliability of Softwarizing Cloud Application Management. In Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI'26), May 2026.
40. H. S. Gunawi, C. Rubio-González, A. C. Arpaci-Dusseau, R. H. Arpaci-Dusseau, and B. Liblit. EIO: Error Handling is Occasionally Correct. In Proceedings of the 6th USENIX Conference on File and Storage Technologies (FAST'08), Feb. 2008.
41. H. S. Gunawi, M. Hao, R. O. Suminto, A. Laksono, A. D. Satria, J. Adityatama, and K. J. Eliazar. Why Does the Cloud Stop Computing? Lessons from Hundreds of Service Outages. In Proceedings of the 7th ACM Symposium on Cloud Computing (SoCC'16), Oct. 2016.
42. H. S. Gunawi, R. O. Suminto, R. Sears, C. Golliher, S. Sundararaman, X. Lin, T. Emami, W. Sheng, N. Bidokhti, C. McCaffrey, G. Gridler, P. M. Fields, K. Harms, R. B. Ross, A. Jacobson, R. Ricci, K. Webb, P. Alvaro, H. B. Runesha, M. Hao, and H. Li. Fail-Slow at Scale: Evidence of Hardware Performance Faults in Large Production Systems. In Proceedings of the 16th USENIX Conference on File and Storage Technologies (FAST'18), Feb. 2018.
43. S. Han, X. Hu, H. Huang, M. Jiang, and Y. Zhao. ADBench: Anomaly Detection Benchmark. In Proceedings of the 36th Annual Conference on Neural Information Processing Systems (NeurIPS'22), Nov. 2022.
44. P. H. Hochschild, P. Turner, J. C. Mogul, R. Govindaraju, P. Ranganathan, D. E. Culler, and A. Vahdat. Cores that don't count. In Proceedings of the ACM SIGOPS 21st Workshop on Hot Topics in Operating Systems (HotOS'21), June 2021.
45. L. Huang, M. Magnusson, A. B. Muralikrishna, S. Estyak, R. Isaacs, A. Aghayev, T. Zhu, and A. Charapko. Metastable Failures in the Wild. In Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI'22), July 2022.
46. P. Huang, C. Guo, L. Zhou, J. R. Lorch, Y. Dang, M. Chintalapati, and R. Yao. Gray Failure: The Achilles' Heel of Cloud-Scale Systems. In Proceedings of the 16th Workshop on Hot Topics in Operating Systems (HotOS'17), pages 150–155, May 2017.

47. R. Isaacs, P. Alvaro, R. Majumdar, K.-K. Muniswamy-Reddy, M. Salamati, and S. Soudjani. Analyzing Metastable Failures. In *Proceedings of the ACM SIGOPS 20th Workshop on Hot Topics in Operating Systems (HotOS'25)*, May 2025.
48. V. Jacob, F. Song, A. Stiegler, B. Rad, Y. Diao, and N. Tatbul. Exathlon: a Benchmark for Explainable Anomaly Detection Over Time Series. *Proceedings of the VLDB Endowment (VLDB'21)*, 14(11):2613–2626, July 2021.
49. S. Jha, S. Cui, S. Banerjee, T. Xu, J. Enos, M. Showerman, Z. T. Kalbarczyk, and R. K. Iyer. Live Forensics for HPC Systems: A Case Study on Distributed Storage Systems. In *Proceedings of the International Conference for High-Performance Computing, Networking, Storage and Analysis (SC'20)*, Nov. 2020.
50. S. Jha, R. R. Arora, Y. Watanabe, T. Yanagawa, Y. Chen, J. Clark, B. Bhavya, M. Verma, H. Kumar, H. Kitahara, N. Zheutlin, S. Takano, D. Pathak, F. George, X. Wu, B. O. Turkkan, G. Vanloo, M. Nidd, T. Dai, O. Chatterjee, P. Gupta, S. Samanta, P. Aggarwal, R. Lee, J. wook Ahn, D. Kar, A. Paradkar, Y. Deng, P. Moogi, P. Mohapatra, N. Abe, C. Narayanaswami, T. Xu, L. R. Varshney, R. Mahindru, A. Sailer, L. Shwartz, D. Sow, N. C. M. Fuller, and R. Puri. ITBench: Evaluating AI Agents across Diverse Real-World IT Automation Tasks. In *Proceedings of the 42nd International Conference on Machine Learning (ICML'25)*, May 2025.
51. J. Kaldor, J. Mace, M. Bejda, E. Gao, W. Kuropatwa, J. O'Neill, K. W. Ong, B. Schaller, P. Shan, B. Viscomi, V. Venkataraman, K. Veeraraghavan, and Y. J. Song. Canopy: An End-to-End Performance Tracing and Analysis System. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP'17)*, Oct. 2017.
52. J.-C. Laprie. Dependable Computing: Concepts, Limits, Challenges. In *Proceedings of the 25th IEEE International Symposium on Fault-Tolerant Computing (FTCS'95)*, June 1995.
53. Y. Lee, J. Kim, J. Kim, H. Cho, J. Kang, P. Kang, and N. Kim. CheckEval: A Reliable LLM-as-a-Judge Framework for Evaluating Text Generation Using Checklists. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP'25)*, Nov. 2025.
54. H. Liu, S. Lu, M. Musuvathi, and S. Nath. What Bugs Cause Production Cloud Incidents? In *Proceedings of the 17th Workshop on Hot Topics in Operating Systems (HotOS'19)*, May 2019.
55. Y. Liu, C. Pei, L. Xu, B. Chen, M. Sun, Z. Zhang, Y. Sun, S. Zhang, K. Wang, H. Zhang, J. Li, G. Xie, X. Wen, X. Nie, M. Ma, and D. Pei. OpsEval: A Comprehensive IT Operations Benchmark Suite for Large Language Models. *arXiv:2310.07637*, June 2025.
56. D. Lo, L. Cheng, R. Govindaraju, P. Ranganathan, and C. Kozyrakis. Heracles: Improving Resource Efficiency at Scale. In *Proceedings of the 42nd Annual International Symposium on Computer Architecture (ISCA'15)*, June 2015.

57. D. Loker. AI vs Human Code Gen Report: AI Code Creates 1.7x More Issues. <https://www.coderabbit.ai/blog/state-of-ai-vs-human-code-generation-report>, 2026.
58. Y. Luo, J. Jiang, J. Feng, L. Tao, Q. Zhang, X. Wen, Y. Sun, S. Zhang, and D. Pei. From Observability Data to Diagnosis: An Evolving Multi-agent System for Incident Management in Cloud Systems. arXiv:2510.24145, Nov. 2025.
59. J. Mao, L. Li, Y. Gao, Z. Peng, S. He, C. Zhang, S. Qin, S. Khalid, Q. Lin, S. Rajmohan, et al. StepFly: Agentic Troubleshooting Guide Automation for Incident Diagnosis. arXiv:2510.10074, Apr. 2026.
60. M. A. Merrill, A. G. Shaw, N. Carlini, B. Li, H. Raj, I. Bercovich, L. Shi, J. Y. Shin, T. Walshe, E. K. Buchanan, J. Shen, G. Ye, H. Lin, J. Poulos, M. Wang, M. Nezhurina, J. Jitsev, D. Lu, O. M. Mastromichalakis, Z. Xu, Z. Chen, Y. Liu, R. Zhang, L. L. Chen, A. Kashyap, J.-L. Uslu, J. Li, J. Wu, M. Yan, S. Bian, V. Sharma, K. Sun, S. Dillmann, A. Anand, A. Lanpouthakoun, B. Koopah, C. Hu, E. Guha, G. H. S. Dreiman, J. Zhu, K. Krauth, L. Zhong, N. Muennighoff, R. Amanfu, S. Tan, S. Pimpalgaonkar, T. Aggarwal, X. Lin, X. Lan, X. Zhao, Y. Liang, Y. Wang, Z. Wang, C. Zhou, D. Heineman, H. Liu, H. Trivedi, J. Yang, J. Lin, M. Shetty, M. Yang, N. Omi, N. Raoof, S. Li, T. Y. Zhuo, W. Lin, Y. Dai, Y. Wang, W. Chai, S. Zhou, D. Wahdany, Z. She, J. Hu, Z. Dong, Y. Zhu, S. Cui, A. Saiyed, A. Kolbeinsson, J. Hu, C. M. Rytting, R. Marten, Y. Wang, A. Dimakis, A. Konwinski, and L. Schmidt. Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces. arXiv:2601.11868, Jan. 2026.
61. J. J. Meza, T. Gowda, A. Eid, T. Ijaware, D. Chernyshev, Y. Yu, M. N. Uddin, R. Das, C. Nachiappan, S. Tran, S. Shi, T. Luo, D. K. Hong, S. Panneerselvam, H. Ragas, S. Manavski, W. Wang, and F. Richard. Defcon: Preventing Overload with Graceful Feature Degradation. In Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI'23), July 2023.
62. N. Palix, G. Thomas, S. Saha, C. Calvès, J. Lawall, and G. Muller. Faults in Linux: Ten Years Later. In Proceedings of the 16th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'11), Mar. 2011.
63. T. Patel and D. Tiwari. CLITE: Efficient and QoS-Aware Co-Location of Multiple Latency-Critical Jobs for Warehouse Scale Computers. In Proceedings of the 2020 IEEE International Symposium on High Performance Computer Architecture (HPCA'20), Feb. 2020.
64. C. Pei, Z. Wang, F. Liu, Z. Li, Y. Liu, X. He, R. Kang, T. Zhang, J. Chen, J. Li, G. Xie, and D. Pei. Flow-of-Action: SOP Enhanced LLM-Based Multi-Agent System for Root Cause Analysis. In Companion Proceedings of the ACM on Web Conference 2025 (WWW'25), May 2025.
65. T. Pelkonen, S. Franklin, J. Teller, P. Cavallaro, Q. Huang, J. Meza, and K. Veeraraghavan. Gorilla: a Fast, Scalable, In-memory Time Series Database. Proceedings of the VLDB Endowment

- (VLDB'15), 8(12):1816–1827, Aug. 2015.
66. B. Schroeder, S. Damouras, and P. Gill. Understanding Latent Sector Errors and How to Protect against Them. In Proceedings of the 8th USENIX Conference on File and Storage Technologies (FAST'10), Feb. 2010.
67. B. H. Sigelman, L. A. Barroso, M. Burrows, P. Stephenson, M. Plakal, D. Beaver, S. Jaspan, and C. Shanbhag. Dapper, a Large-Scale Distributed Systems Tracing Infrastructure. Technical Report dapper-2010-1, Google, Inc., Apr. 2010.
68. X. Sun, R. Cheng, J. Chen, E. Ang, O. Legunsen, and T. Xu. Testing Configuration Changes in Context to Prevent Production Failures. In Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI'20), Nov. 2020.
69. X. Sun, W. Luo, J. T. Gu, A. Ganesan, R. Alagappan, M. Gasch, L. Suresh, and T. Xu. Automatic Reliability Testing for Cluster Management Controllers. In Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI'22), July 2022.
70. P. Tang, S. Tang, H. Pu, Z. Miao, and Z. Wang. MicroRCA-Agent: Microservice Root Cause Analysis Method Based on Large Language Model Agents. arXiv:2509.15635, Sept. 2025.
71. Y. Tian, Y. Liu, Z. Chong, Z. Huang, and H.-A. Jacobsen. GALA: Can Graph-Augmented Large Language Model Agentic Workflows Elevate Root Cause Analysis? arXiv:2508.12472, Aug. 2025.
72. B. Treynor, M. Dahlin, V. Rau, and B. Beyer. The Calculus of Service Availability. Communications of the ACM (CACM), 60(9):42–47, Aug. 2017.
73. K. Veeraraghavan, J. Meza, S. Michelson, S. Panneerselvam, A. Gyori, D. Chou, S. Margulis, D. Obenshain, S. Padmanabha, A. Shah, Y. J. Song, and T. Xu. Maelstrom: Mitigating Datacenter-level Disasters by Draining Interdependent Traffic Safely and Efficiently. In Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI'18), Oct. 2018.
74. H. Wang, Q. Mang, A. Cheung, K. Sen, and D. Song. How We Broke Top AI Agent Benchmarks: And What Comes Next. <https://rdi.berkeley.edu/blog/trustworthy-benchmarks-cont/>, 2026.
75. Y. Wang, G. Yu, H. Huang, Z. Wang, Y. Huang, P. Chen, and M. R. Lyu. Cloud-OpsBench: A Reproducible Benchmark for Agentic Root Cause Analysis in Cloud Systems. arXiv:2603.00468, Feb. 2026.
76. Z. Wang, Z. Liu, Y. Zhang, A. Zhong, J. Wang, F. Yin, L. Fan, L. Wu, and Q. Wen. RCAgent: Cloud Root Cause Analysis by Autonomous Agents with Tool-Augmented Large Language Models. In Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM'24), Oct. 2024.

77. J. Xu, Q. Zhang, Z. Zhong, S. He, C. Zhang, Q. Lin, D. Pei, P. He, D. Zhang, and Q. Zhang. OpenRCA: Can Large Language Models Locate the Root Cause of Software Failures? In Proceedings of the 13th International Conference on Learning Representations (ICLR'25), Jan. 2025.
78. T. Xu, J. Zhang, P. Huang, J. Zheng, T. Sheng, D. Yuan, Y. Zhou, and S. Pasupathy. Do Not Blame Users for Misconfigurations. In Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP'13), Nov. 2013.
79. T. Xu, X. Jin, P. Huang, Y. Zhou, S. Lu, L. Jin, and S. Pasupathy. Early Detection of Configuration Errors to Reduce Failure Damage. In Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI'16), Nov. 2016.
80. S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing Reasoning and Acting in Language Models. In Proceedings of the 11th International Conference on Learning Representations (ICLR '23), May 2023.
81. D. Yuan, H. Mai, W. Xiong, L. Tan, Y. Zhou, and S. Pasupathy. SherLog: Error Diagnosis by Connecting Clues from Run-Time Logs. In Proceedings of the 15th International Conference on Architecture Support for Programming Languages and Operating Systems (ASPLOS'10), Mar. 2010.
82. E. Zhai, A. Chen, R. Piskac, M. Balakrishnan, B. Tian, B. Song, and H. Zhang. Check before You Change: Preventing Correlated Failures in Service Updates. In Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI'20), Feb. 2020.
83. L. Zhang, T. Jia, K. Wang, W. Hong, C. Duan, M. He, and Y. Li. Adaptive Root Cause Localization for Microservice Systems with Multi-Agent Recursion-of-Thought. arXiv:2508.20370, Aug. 2025.
84. L. Zhang, Y. Zhai, T. Jia, C. Duan, M. He, L. Pan, Z. Liu, B. Ding, and Y. Li. MicroRemed: Benchmarking LLMs in Microservices Remediation. arXiv:2511.01166, Nov. 2025.
85. W. Zhang, H. Guo, J. Yang, Z. Tian, Y. Zhang, Y. Chaoran, Z. Li, T. Li, X. Shi, L. Zheng, and B. Zhang. mABC: Multi-Agent Blockchain-inspired Collaboration for Root Cause Analysis in Micro-Services Architecture. In Findings of the Association for Computational Linguistics (EMNLP'24), Nov. 2024.
86. X. Zhang, Q. Wang, M. Li, Y. Yuan, M. Xiao, F. Zhuang, and D. Yu. TAMO: Fine-Grained Root Cause Analysis via Tool-Assisted LLM Agent With Multi-Modality Observation Data in Cloud-Native Systems. IEEE Transactions on Services Computing (TSC), 18(6):4221–4233, Nov. 2025.
87. Y. Zhang, K. Rodrigues, Y. Luo, M. Stumm, and D. Yuan. The Inflection Point Hypothesis: A Principled Debugging Approach for Locating the Root Cause of a Failure. In Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP'19), Oct. 2019.

88. Y. Zhang, U. Paul, M. d’Amorim, and A. Rahman. Configuration Defects in Kubernetes. arXiv:2512.05062, Dec. 2025.
89. L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In Proceedings of the 37th Annual Conference on Neural Information Processing Systems (NeurIPS’23), Dec. 2023.
90. Z. Zhong, R. Fu, M. Ma, S. Zhang, Y. Sun, C. Bansal, and D. Pei. LLM-Enhanced Failure Localization in Microservices: Integrating Multi-Modal Data and Expert Interpretation. IEEE Transactions on Services Computing (TSC), pages 1–14, Mar. 2026.
91. X. Zhou, X. Peng, T. Xie, J. Sun, C. Xu, C. Ji, and W. Zhao. Benchmarking Microservice Systems for Software Engineering Research. In Proceedings of the 40th International Conference on Software Engineering: Companion Proceedings (ICSE ’18), May 2018.
92. Y. Zhu, T. Jin, Y. Pruksachatkun, A. Zhang, S. Liu, S. Cui, S. Kapoor, S. Longpre, K. Meng, R. Weiss, F. Barez, R. Gupta, J. Dhamala, J. Merizian, M. Giulianelli, H. Coppock, C. Ududec, J. Sekhon, J. Steinhardt, A. Kellermann, S. Schwettmann, M. Zaharia, I. Stoica, P. Liang, and D. Kang. Establishing Best Practices for Building Rigorous Agentic Benchmarks. arXiv:2507.02825, July 2025.

A 诊断评测清单

表 6 列出了诊断预言机 (§2.5) 所用的完整清单。每个问题需要一个是否回答；LLM 评测者针对每个问题返回支持证据和置信度（高/中/低）。维度权重和通过阈值在一个 YAML 文件中设置（当前默认值： $w = 1/3$ ）。

表 6: 完整的诊断评测清单。每个问题由 LLM 评测者回答是/否。维度内得分公式和聚合得分公式见 §2.5。

故障定位 (Fault Localization, $w = 1/3$)	
D1-Q1	诊断是否指出了与真值所认定的故障起源相同的服务、deployment、pod、节点或基础设施组件? (评测者提示: 与故障规约 (spec) 中的目标组件和目标资源类型进行比对。)
D1-Q2	诊断是否正确地把故障起源与真值中提及的任何次级或级联失效点区分开来? (评测者提示: 检查诊断是否指向根因组件, 而非下游受害者。)
D1-Q3	诊断是否避免把一个健康的组件误判为故障起源? (评测者提示: 核实被诊断的组件与故障规约的目标组件一致。)
故障刻画 (Fault Characterization, $w = 1/3$)	
D2-Q1	诊断是否识别出了与真值描述相同的注入机制 (例如错误的网络端口、缺失的环境变量、错误的容器镜像、错误的 selector、内存限制)? (评测者提示: 与结构化规约中的故障机制和注入器方法进行匹配。)
D2-Q2	诊断是否包含来自注入逻辑的具体变异细节 (例如环境变量、配置值、网络端口、selector、容器镜像标签、资源限制)? (评测者提示: 将具体声明与参数以及注入器方法所隐含的目标变异进行比对。)
D2-Q3	诊断是否避免把故障归因于一个不正确或不相关的故障类型? (评测者提示: 检查诊断是否与问题类别、注入器方法或注入的参数值相冲突。)
范围精确性 (Scope Precision, $w = 1/3$)	
D3-Q1	诊断是否避免归咎于真值中未被认定为对故障有贡献的组件? (评测者提示: 检查是否过度归因: 诊断不应归咎于无关组件。)
D3-Q2	诊断是否包含真值中列出的、对故障有贡献或受其影响的所有组件? (评测者提示: 检查是否归因不足: 所有真值组件都应被指出。)
D3-Q3	诊断对影响或症状的描述是否与真值所陈述的一致? (评测者提示: 将所陈述的影响与故障规约中的机制、参数和目标组件进行比对。)

B 工程实践

在构建 SREGym 的过程中, 我们做出了若干工程决策, 我们相信它们对任何旨在评测自主 SRE 智能体的基准测试都很重要。我们将其记录为给未来 SRE 基准测试开发者的建议, 附上每条背后的理由, 并在有用之处给出我们在现有基准测试 [32, 50, 75, 84] 中观察到的偏离做法的示例。

实时环境的益处。一种自然的设计是在某一时刻捕获环境遥测数据，并把它作为静态产物呈现给智能体，其动机是希望从评测中消除不确定性 [75]。然而，静态产物存在根本性局限。噪声和不确定性是 SRE 智能体必须身处其中的生产环境的内在属性，在生产环境中，信号往往是不完整的，遥测可能与故障产生竞态，并且环境状态在智能体采取动作的同时还在演化。快照把任务退化为静态日志分诊，并消除了该基准测试声称要衡量的运维技能：形成假设、发出探测、并观察实时环境如何响应。快照还瓦解了排障的交互本质。真实的事事故响应是迭代且按时间顺序进行的：一名 SRE 工程师/智能体发出一条命令，等待其生效，观察系统状态，并据此决定下一步。因此，SREGym 采用实时系统环境，使智能体在镜像生产系统的嘈杂、演化环境中接受评测。

对智能体架构不作限制。SRE 智能体的空间既包括带有领域专用工具的多智能体系统（例如 Stratus [31]），也包括通用编码智能体（例如 Claude Code 和 Codex）。例如，AIOpsLab [32] 要求受测智能体在一个由编排器（orchestrator）中介的 ReAct [80] 循环中交互，该编排器解析每一个动作并暴露一组固定的函数签名。非 ReAct 架构构建的智能体需要被移植或集成后才能在 AIOpsLab 上评测。SREGym 把其智能体接口保持到最小：只要求 `submit()` 调用。这种解耦使我们能够用同一个基准测试框架，在同一组问题上评测像 Stratus、Claude Code 和 Codex 这样架构迥异的智能体。

用于同步和协调事件的可编程运行时。基准测试运行时必须有足够的表达力，能够跨注入器、预言机和智能体的动作来调度和同步事件。SREGym 的两类问题使这一要求变得具体。亚稳态失效（见 §D.1）把一个自我维持的应用触发因素（例如错误配置的 GOGC 值或重试风暴配置）与一个基础设施约束（例如紧绷的命名空间内存配额）配对；这两个组件必须以正确的时序和顺序注入，才能驱动系统进入亚稳态，而缓解预言机必须持续观测环境，以把瞬态恢复与复发区分开来。噪声模拟需要一个独立的并发循环，在目标故障处于活动状态时按其自己的调度计划注入瞬态事件，从而使智能体看到干扰证据与目标证据同时出现的混合体。像 ITBench [50] 那样建立在 Ansible Playbook 之类基底上的基准测试无法直接表达这些协调模式，往往把缺失的功能推给临时的 shell 脚本，这使得对时序敏感的故障和并发噪声难以控制和复现。SREGym 作为一个 Python 服务运行，在单个进程中拥有故障调度、噪声调度和预言机探测，因此问题开发者可以在问题定义中表达对时序敏感的行为，而不必借助外部、间接的脚本。

避免误用混沌工程工具。诸如 Chaos Mesh [1] 和 Chaosblade [13] 之类的混沌工程工具在其设计用途上很有价值：扰动一个运行中的系统，以测试应用在意外故障下的韧性（resiliency）。从设计上看，这些工具注入的是症状（被杀死的 pod、被丢弃的数据包、被限流的 CPU、延迟尖峰），而非缺陷。没有底层故障供 SRE 智能体去发现，唯一有效的“缓解”就是停止混沌注入器，而这并不是一项应当奖励智能体去学习的运维技能。若干近期基准测试把韧性测试与运维评测混为一谈：MicroRemed [84] 的故障完全来自 Chaos Mesh；Cloud-OpsBench [75] 在其 Kubernetes 错误配置之外还借助 Chaosblade；ITBench [50] 虽然在其他情况下通过直接操作 Kubernetes 来注入故障，但在其约 16%（36 个中的 6 个）的 SRE 场景中包裹了一个 Chaos Mesh 调度。对于这些场景，没有缺陷供智能体修复，唯一能“缓解”问题的动作是停止混沌工程工具或删除其调度。SREGym 通过直接操作 Kubernetes、系统调用级的 eBPF 探针和 operator 误操作来注入故障，而非通过症状性扰动，因此每个 SRE 问题都有底层缺陷供智能体诊断和解决。

对奖励作弊的防护。 AI 智能体可能利用基准测试基础设施来虚高得分，而不解决底层任务 [74]。在 SRE 基准测试中，最直接的表现是智能体发现并禁用故障注入服务，而不去推理实际的故障。AIOpsLab [32] 和 ITBench [50] 都未能恰当地防护此类奖励作弊：它们的故障注入器作为可识别的 pod 运行在智能体所检查的同一环境中。第二种利用方式是把清除告警当作成功信号：Stratus 论文 [31] 报告，ITBench 的 18 个缓解问题中有 8 个（44%）可以被一个通用的 pod 重启循环“解决”——在 pod 被重启后，故障注入器跟丢了该 pod，告警随之清除，智能体便被记为缓解成功（尽管它对实际缺陷未采取任何动作）。SREGym 把其故障注入平面隐藏在一个代理（proxy）之后，受测智能体对其不可见，并使用基于状态的缓解预言机来探测实时环境健康状况，而非依赖告警抑制。

C 组合覆盖度细节

我们提供 §2.4 开头所报告的组合覆盖度背后的详细分解。表 7 按故障原语所兼容的目标组件类别（以下简称“目标”）对 SREGym 的故障原语进行分组，并报告在 139 个 pod 的注入目标空间（外加用于硬件级故障的三个工作节点）中，每个类别的可行（故障，目标）配对数量。

表 7: 故障-目标兼容性类别。“# Faults”指每个类别中故障原语的数量；由于一个故障原语可以与多个目标类别兼容（例如缺失环境变量同时影响 MongoDB 和非 MongoDB 的 pod），因此该列加总不等于原语总数。“Pairs”是可行（故障，目标）组合的计数，合计 3,623。

类别	# Faults	兼容目标	Pairs
通用 Kubernetes 级	25	139 个 pod	3,475
依赖存储的	5	6 个挂载 PVC 的 pod	30
DaemonSet 级	1	3 个 daemonset	3
Operator 级	6	5 个 Kubernetes 应用	30
MongoDB 专用	4	18 个 MongoDB pod	72
Valkey 专用	2	1 个 pod	2
应用层错误配置	1	2 个服务	2
节点/内核	3	3 个工作节点	9
合计			3,623

我们在 §3 中评测所针对的 90 个策划问题，只使用了 3,623 个可行（故障，目标）配对中的 2.5%。噪声模拟和多故障组合进一步使组合数量成倍增长。同样的组合结构使 SREGym 成为强化学习 roll-out 的一个天然环境，其中每个（故障，目标）配对都成为针对一个实时、类生产系统环境的一个回合（episode） (§5)。

D 案例研究

D.1 亚稳态失效

SREGym 包含建模亚稳态失效的问题，其中一个临时触发因素把系统推入一个自我维持的退化状态，即使触发因素被移除该状态仍持续存在 [29, 45, 47]。每个问题都是一个复合故障：一个应用层触发因素（例如放大流量的错误配置重试策略，或强制频繁垃圾回收的运行标志）与一个把系统驱入易损状态的基础设施约束（例如限制 CPU 或内存分配的资源配额或限制）配对。该约束不产生任何错误、也不产生任何失败的追踪；它只能通过显式检查部署配置才能被发现。触发因素把系统从易损状态驱入自我维持的亚稳态：系统性能退化，却无明显的失停症状。仅修复触发因素并不能缓解症状：智能体必须推理触发因素与持续退化状态之间的关系，然后通过移除基础设施级约束并重启应用（给它一个干净的起点）来缓解问题。

在我们的评测中，智能体能够通过追踪分析和部署检查可靠地诊断出应用层触发因素，但几乎从不发现基础设施约束，因为它不产生任何可观测的症状。在 Codex 确实发现基础设施资源约束的那一次运行中，它把整个失效都归因于该约束，并否定了应用层错误配置。在任何情况下，智能体都未能识别出触发因素与约束之间的相互作用。

我们观察到智能体有时确实能修复亚稳态行为。智能体修复了错误配置并重启了受影响的服务；这恢复了系统并清理了被叠加的流量，从而使系统能够恢复正常执行。当智能体未能缓解亚稳态行为时，往往是因为智能体被表层症状分散了注意力，并把失效归因于一个完全不同的故障类型。例如，在 `gc_capacity_degradation` 问题的一次运行中——其中一个过度激进的低 GOGC 设置强制在一个容量受限的命名空间内对所有工作负载频繁进行垃圾回收——智能体根本没有检查 GOGC。相反，智能体固执于分布式追踪所显示的、前端与 `profile-service` 之间约 300 ms 的间隙，而服务端的 `GetProfiles` 处理函数在微秒级内完成。仅凭这一模式，智能体便断定该延迟一定存在于网络路径中，并提交了一个根因：一条在 `profile-service` pod 的 `veth` 接口上每个方向注入约 150 ms 延迟的 `tc netem` 规则。这一不正确的诊断犯了两个独立的错误：（1）故障类型为网络延迟注入，以及（2）范围为单个服务。结果，智能体提出的缓解针对一条不存在的网络规则，并且从未重启工作负载以清除亚稳态垃圾回收循环。

D.2 硬件故障

SREGym 中的 `latent_sector_error` 问题向一个物理节点的存储设备注入硬件故障。一块存储磁盘会在文件读取系统调用（例如 `pread()`）上返回间歇性错误。部署在该节点上的 MongoDB 数据库会因无法读取与存储引擎相关的元数据文件而崩溃，并报出 “`read: input/output error`”。

我们的评测表明，智能体难以诊断出与硬件相关的根因。在 Stratus、Claude Code 和 Codex 各三次、未进行噪声模拟的运行中，没有一次产生的诊断得分超过 0.22，并且 D2（故障刻画）在每次运行中都得到 0 分，表明它们的诊断从未触及系统的正确组件。例如，智能体观察到 I/O 错误来自 `memcached deployment`（它从 MongoDB deployment 读取数据），并错误地把失效归因于 `memcached`

deployment 的连接被丢弃。智能体也归咎于用户工作负载压垮了 memcached 进程，导致其重置连接。最后，它们归咎于微服务应用未能优雅地处理 I/O 错误。这些结论从未提及底层硬件可能存在缺陷。

这一问题暴露了一个具体的弱点：缺乏对底层硬件如何影响其上部署应用的理解。智能体把暴露出来的 I/O 错误当作错误处理不当的证据，却从不建议磁盘级诊断——而后者正是 SRE 应对硬件故障的标准做法。

D.3 诊断故障时的贪心方法

在我们的整个评测中，一个反复出现的智能体失败模式是：智能体在诊断和缓解时对环境中的任何首先观察到的异常采取贪心方法。智能体首先在环境中遇到一个被怀疑但无关的异常，并提交一份不成熟的诊断，这份诊断随后会误导缓解阶段。这些被怀疑的异常既可能是与目标故障一同运行的被注入噪声，也可能是预先存在的应用特征，例如应用容器中较低的内存限制。

我们观察到，无论智能体抓住的是哪一个异常，这一模式都是一致的。图 1 展示了一个使用 `missing_env_variab` 问题的示例，其中实际故障是应用前端组件上被丢弃的一个环境变量。在 Stratus 和 Claude Code 的所有运行中，两个智能体都从未检查过前端组件。Stratus 抓住了另一个具有较低内存限制的 deployment，而 Claude Code 则聚焦于 Jaeger 追踪基础设施。两者都找到了看似合理的问题，把它们诊断为根因，并且从未调查实际有故障的组件。这种贪心方法影响了缓解的有效性。在固执于内存限制之后，智能体用 `kubectl patch` 或类似的编辑提高了它，但显然这一修复并未解决肇事故障。

一个更有意思（或者说更令人惋惜）的案例发生在智能体确实遇到了肇事故障的证据，却仍锚定于其初始假设时。在 `valkey_auth_disruption` 问题中，故障是在运行时为 Valkey 数据库设置了一个无效密码。依赖它的微服务组件因无法对数据库进行认证而崩溃。智能体常常把崩溃归因于内存不足，并指出这些容器有较低的内存限制。然而，智能体也观察到 `init` 容器的 TCP 健康检查成功，而应用层的数据库连接失败。这一差异与认证失败相符，而非资源耗尽。在一次运行中，智能体多次把设置密码的命令（`requirepass`）作为候选假设提及，但最终仍退回到内存解释。

这一案例研究暴露了一个具体的弱点：智能体把第一个看似合理的异常当作停止准则。一旦写下了一个候选根因，在缓解阶段，模型便把后续证据解读为支持该根因，而不生成相互竞争的假设。相比之下，一名人类 SRE 能够形成多个假设并并发地进行调查，直到找到真正的根因。

E 工具使用细节

我们对 §3.4 中报告的工具使用进行了详细分析。我们把智能体轨迹中的每一次工具调用归入以下类别：

- **kubectl (读)**: 用于对系统状态进行只读检查（例如 `get`、`logs`、`describe`、`top`、`auth`）。
- **kubectl (写)**: 用于改变系统状态（例如 `patch`、`rollout`、`set`、`run`、`delete`、`apply`、`scale`、`port-forward`）。

- **可观测性 (Observability)**: 用于查询指标、追踪和服务图。Stratus 使用专用的智能体工具; Claude Code 和 Codex 使用 curl 查询 Prometheus、Jaeger 或 Loki 端点。
- **网络 (Network)**: 用于连通性探测 (例如向应用端点发起 curl、ss、netstat、nc、ping)。
- **等待/休眠 (Wait/sleep)**: 用于有意的暂停 (例如 Stratus 的 wait_tool 和其他智能体的 sleep)。
- **Shell**: 用于通用工具 (例如 cat、ls、grep、python、sed、export)。
- **其他 (Others)**: 例如智能体框架原生工具 (例如 Claude Code 的 Read、Agent、WebSearch; Kimi 的 thinking 工具)。

提交 (Submission) 调用从所有计数中排除。表 8 展示了每种评测配置下的工具调用类别分解。表 9 展示了系统环境中读命令与写命令的比率。表 10 和表 11 展示了 kubectl 中执行最多的子命令。所有平均值的计算方式是: 先对每个问题在各次运行间取均值, 再在各问题间取均值。

表 8: 工具调用类别分解。

智能体	模型	噪声	kctl(读)	kctl(写)	可观测性	网络	等待	Shell	其他
Stratus	Sonnet 4.6	□	65.8	11.9	16.6	0.0	5.6	0.0	0.0
Stratus	Sonnet 4.6	■	71.8	8.3	14.5	0.0	5.1	0.0	0.2
Stratus	K2.5	□	71.6	8.5	14.6	0.0	4.6	0.0	0.7
Stratus	K2.5	■	72.0	8.1	14.8	0.0	4.4	0.0	0.6
Claude Code	Sonnet 4.6	□	61.5	4.6	2.4	8.3	4.2	16.5	2.5
Claude Code	Sonnet 4.6	■	64.8	4.5	2.1	8.5	2.8	14.0	3.4
Codex	GPT-5.4	□	60.2	7.7	2.4	8.7	2.7	18.4	0.1
Codex	GPT-5.4	■	61.6	7.6	2.7	7.4	2.7	17.8	0.2

表 9: kubectl 读/写分析。Ratio 为读总数除以写总数。“Reads → First Write” 是在系统状态首次发生变更之前的只读 kubectl 命令的平均数量, 先按问题取均值再跨问题取均值。

智能体	模型	噪声	比率	首次写之前的读数
Stratus	Sonnet 4.6	□	5.5:1	22.7
Stratus	Sonnet 4.6	■	8.6:1	24.2
Stratus	K2.5	□	8.4:1	24.8
Stratus	K2.5	■	8.9:1	27.6
Claude Code	Sonnet 4.6	□	13.5:1	21.3
Claude Code	Sonnet 4.6	■	14.5:1	23.9
Codex	GPT-5.4	□	7.9:1	18.9
Codex	GPT-5.4	■	8.1:1	21.2

表 10: 每个智能体执行最多的前 3 个 kubectl 读子命令（占所有只读 kubectl 调用的百分比）。

智能体	模型	噪声	第 1	第 2	第 3
Stratus	Sonnet 4.6	□	get (64)	logs (18)	describe (13)
Stratus	Sonnet 4.6	■	get (65)	logs (17)	describe (14)
Stratus	K2.5	□	get (69)	logs (20)	describe (9)
Stratus	K2.5	■	get (67)	logs (21)	describe (10)
Claude Code	Sonnet 4.6	□	get (59)	logs (23)	describe (10)
Claude Code	Sonnet 4.6	■	get (62)	logs (19)	describe (11)
Codex	GPT-5.4	□	get (60)	logs (24)	exec (7)
Codex	GPT-5.4	■	get (57)	logs (25)	exec (8)

表 11: 每个智能体执行最多的前 5 个 kubectl 写子命令（占所有变更性 kubectl 调用的百分比）。

智能体	模型	噪声	第 1	第 2	第 3	第 4	第 5
Stratus	Sonnet 4.6	□	patch (39)	set (19)	run (14)	delete (9)	rollout (8)
Stratus	Sonnet 4.6	■	patch (41)	set (23)	rollout (9)	delete (7)	create (6)
Stratus	K2.5	□	patch (41)	set (15)	create (13)	rollout (11)	delete (6)
Stratus	K2.5	■	patch (40)	set (13)	delete (11)	create (10)	rollout (9)
Claude Code	Sonnet 4.6	□	rollout (35)	patch (32)	run (9)	delete (7)	apply (5)
Claude Code	Sonnet 4.6	■	rollout (40)	patch (29)	delete (11)	run (7)	apply (4)
Codex	GPT-5.4	□	rollout (33)	patch (20)	run (19)	port-fwd (12)	delete (5)
Codex	GPT-5.4	■	rollout (29)	run (21)	patch (17)	port-fwd (11)	delete (11)

表 12: 缓解重试分析（仅 Stratus 变体）。缓解尝试总次数和缓解读次数以初始诊断正确（D）或不正确（¬D）为条件。

智能体	模型	噪声	尝试 D	尝试 ¬D	缓解读 D	缓解读 ¬D
Stratus	Sonnet 4.6	□	1.88	3.82	17.3	66.2
Stratus	Sonnet 4.6	■	1.55	1.61	13.2	21.6
Stratus	K2.5	□	1.79	1.90	19.3	26.4
Stratus	K2.5	■	1.56	1.51	16.4	21.2

F 总 Token 数与端到端成功率分析

图 6 针对每种（智能体，模型，噪声）配置，绘制了每次运行的平均总 token 数与端到端成功率的关系。该图区分出两种区间。Stratus 平均每次运行消耗 0.53M token（范围 0.41–0.81M），而 Claude Code 使用 1.59M (3.0×)，Codex 使用 1.93M (3.6×)，这与 Stratus 的多智能体架构相一致——该架构预处理可观测性数据，仅以过滤后的子集提示底层 LLM。编码智能体则把原始可观测性数据流拉入 LLM 上下文，这虚高了 token 用量却没有带来端到端成功率的相应提升。

在这组实验中，token 花费并不能预测成功。最高的端到端成功率由 Claude Code 在未注入噪声的环境中取得，但若干 Stratus 配置与之仅相差几个百分点，而它们只花费了一小部分 token。Codex 在所有配置中花费的 token 最多，但端到端成功率落后于 Claude Code，这表明在 SREGym 上，边际 token 并不能换来额外的成功。

噪声会降低每个智能体的端到端成功率（每种颜色的噪声标记都位于其干净对应标记之下），但噪声带来的 token 成本很小。编码智能体在噪声下确实会因摄入额外的异常信号而多花费一些 token，而 Stratus 的 token 用量在本质上与噪声无关，因为其预处理层会在额外信号到达 LLM 之前就将其吸收。

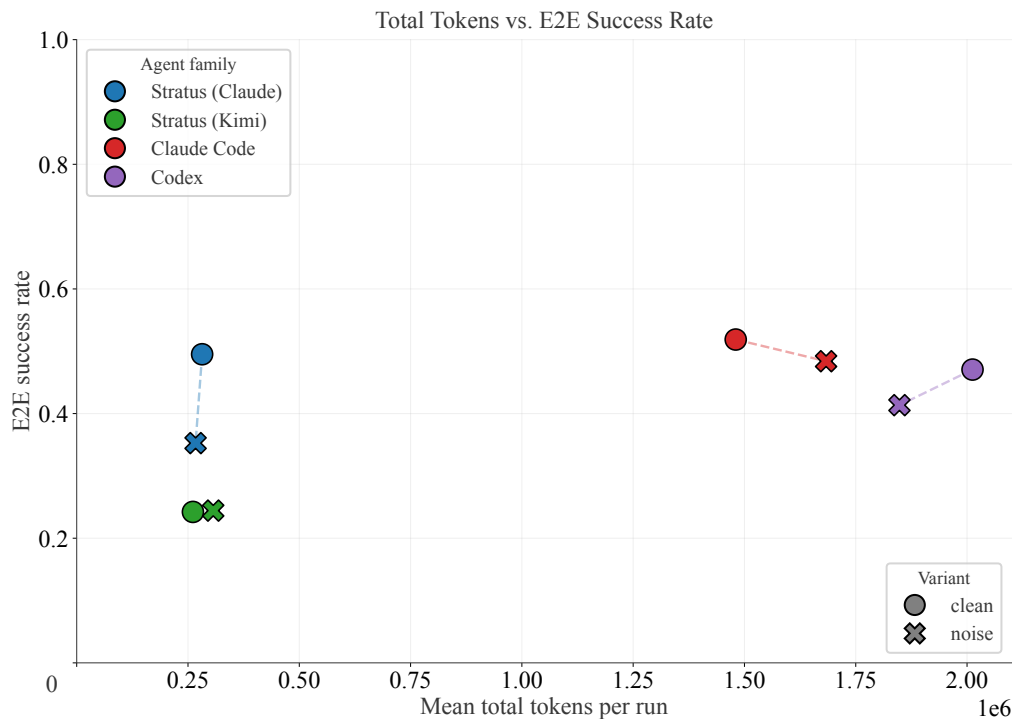


图 6: 针对每种（智能体，模型，噪声）配置，每次运行的平均总 token 数与端到端成功率 ($P(D \wedge M)$) 的关系。圆圈表示干净条件；叉号表示噪声条件。Stratus 配置占据低 token / 中等成功率区间，而 Claude Code 和 Codex 占据高 token 区间，却没有端到端成功率的相应提升。

G 更广泛的影响

SREGym 是一个用于在站点可靠性工程 (SRE) 上评测 AI 智能体的基准测试。我们讨论本工作所预见的正面和负面社会影响, 以及相应的缓解措施。

正面影响。最直接的正面影响是提升 AI 辅助的计算系统运维的可靠性。生产故障是服务宕机、经济损失以及工程师/运维人员倦怠的一个主要原因。越来越多的 AI 产品被推向市场以实现自主 SRE [11, 16–18]。如果没有真实的、面向缓解的基准测试, 关于这些技术和产品的声称就难以核实。SREGym 旨在评测 AI 智能体迈向自主 SRE 的能力。它在整个系统栈 (应用、平台、操作系统、硬件) 中注入故障, 把它们与并发噪声组合以建模真实世界的生产环境, 并针对系统状态验证缓解。通过让在真实、高保真 SRE 问题上评测 SRE 智能体变得更容易, 我们旨在抬高 AI/智能体化 SRE 产品必须跨过的经验门槛, 并降低未经充分测试的智能体被授予生产访问权限的几率。第二项正面影响是为研究提供开放基础设施。可组合的 API、故障与噪声注入器以及基于 MCP 的智能体接口被发布出来, 使其他研究人员能够把 SREGym 扩展到新应用、新故障类别和新智能体架构, 而无需重建脚手架。事实上, 我们已经听闻许多同行在他们的研究项目中使用 SREGym。

负面影响与缓解。我们考虑了三类潜在的负面影响。第一, 自动化替代: 有能力的 AI SRE 智能体可能减少对入门级运维工作的需求, 正如 AI 编码助手正在重塑软件开发岗位一样。然而, 我们的结果表明这一风险目前仍较为遥远: 即便是前沿智能体在 SREGym 的真实失效场景上也举步维艰 (见 §3 和 §D)。第二, 故障注入机制的滥用: SREGym 的注入器原则上可能被改用于在用户所控制的生产系统中造成危害。我们指出, 我们所用的所有注入原语都是标准的、广泛可用的工具; 因此, SREGym 并未引入超出这些工具已经暴露之外的新攻击能力。该基准测试需要合法的集群凭据才能运行, 使用这些原语发动一次真实攻击并不比直接使用底层工具更容易。第三, 对基准测试数字的过度依赖: 一个高 SREGym 得分是智能体可安全授予生产访问权限的必要但不充分证据。第 H 节明确列举了我们的得分应在何种假设和范围下被解读, 包括小于生产规模的应用、一个简化的噪声模型, 以及一小组受测的智能体-模型组合。我们鼓励用户把 SREGym 结果解读为智能体能够从中恢复的失效模式的下界, 而非它在真实世界生产部署中将遇到的失效模式的上界。

H 局限性

SREGym 存在一些局限, 它们影响其结果应如何被解读和扩展。

预言机方差。诊断预言机依赖一个 LLM 评测者, 这引入了纯程序化检查所没有的方差来源。我们通过把 LLM 评测者约束到一个带有逐问题回答的结构化评分细则 (附录 A), 并通过报告逐维度的分解 (以便读者看到判断在何处稳定、在何处嘈杂) 来缓解这一点。尽管具有很强的经验可靠性 (见表 2), 它仍只是大规模评测自然语言诊断结果的一个当前最优的近似。

噪声建模。SREGym 注入的噪声是对真实世界生产扰动的一种简化。我们按调度计划注入瞬态 pod 崩溃和资源压力, 这捕捉到了一种常见模式 (繁忙系统中例行的 pod 流转), 但并未建模所有生产噪声来源, 例如高方差的流量异常 [61]、部分网络分区 [25], 或由资源逐渐耗尽所导致的缓慢性能退化

[34]。在 SREGym 的噪声模型上表现良好的智能体，仍可能被我们尚未涵盖的噪声模式所出乎意料。

系统规模。 SREGym 部署的应用比主要云服务提供商的生产系统要小得多。我们最大的应用 Train Ticket [91] 有 40 个微服务，而 Uber、Netflix 和 Meta 等公司的生产部署通常运行着数千个相互交互的服务 [37, 61]。规模同时影响诊断（智能体面对更少的候选服务需要调查）和缓解评测（更少的跨服务依赖需要推理）。SREGym 上的结果可能无法线性地外推到规模大几个数量级的系统。

环境范围。 SREGym 针对云原生、基于 Kubernetes 的部署，这是现代生产系统的主导平台，但并非唯一平台。运行在单体（monolithic）部署或边缘部署上的工作负载有着不同的失效模式，这些是 SREGym 目前未予演练的。扩展到这些环境需要新的故障注入器和可观测性集成，而可组合架构正是为支持这一点而设计的。

评测中的智能体覆盖度。 受预算限制，我们在评测中只能覆盖三个智能体（Stratus、Claude Code、Codex），分别搭配三个前沿模型（Claude Sonnet 4.6、Kimi K2.5、GPT-5.4）。相对于可能的智能体架构和 LLM 主干的空间，这是一个很小的样本，因此我们关于通用编码智能体与专用 SRE 智能体的结论相应地是初步的。我们发布该基准测试和评分流水线，以便社区能够评测更多的智能体-模型组合，并在相同的预言机下报告结果。我们承诺支持此类工作并维护排行榜。

I 逐问题端到端结果

图 7 和图 8 分别展示了在环境中注入噪声和未注入噪声两种情况下，每个智能体的逐问题端到端（E2E）成功率。问题（行）按各智能体间的平均端到端成功率排序；智能体（列）按总体端到端成功率排序。单元格值 $k/3$ 表示三次运行中有 k 次既做到了正确诊断又做到了成功缓解。

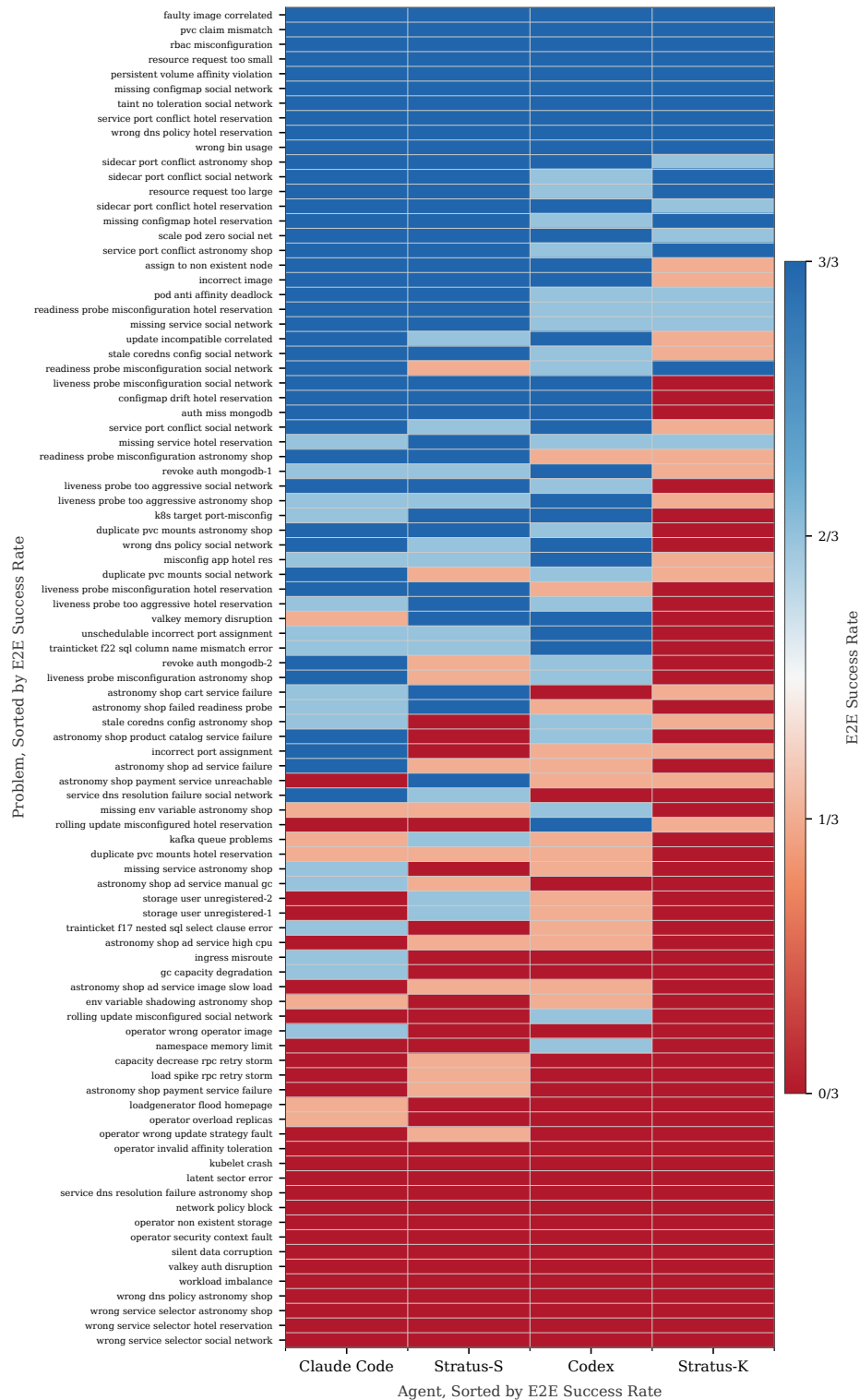


图 7: 未注入噪声时的逐问题端到端成功率。智能体列按端到端成功率排序: Claude Code、Stratus-S、Codex、Stratus-K。问题行按各智能体间的端到端成功率排序, 涵盖如 faulty image correlated、pvc claim mismatch、rbac misconfiguration、resource request too small、missing configmap social network、service port conflict hotel reservation、incorrect image、pod anti affinity deadlock、auth miss mongodb、k8s target port-misconfig、valkey memory disruption、trainticket f22 sql column name mismatch error、missing env variable astronomy shop、kafka queue problems、gc capacity degradation、namespace memory limit、capacity decrease rpc retry storm、load spike rpc retry storm、loadgenerator flood homepage、kubelet crash、latent sector error、network policy block、silent data corruption、valkey auth disruption、workload imbalance、wrong service selector social network 等约 90 个问题。颜色刻度从 0/3 到 3/3 表示 E2E 成功率。

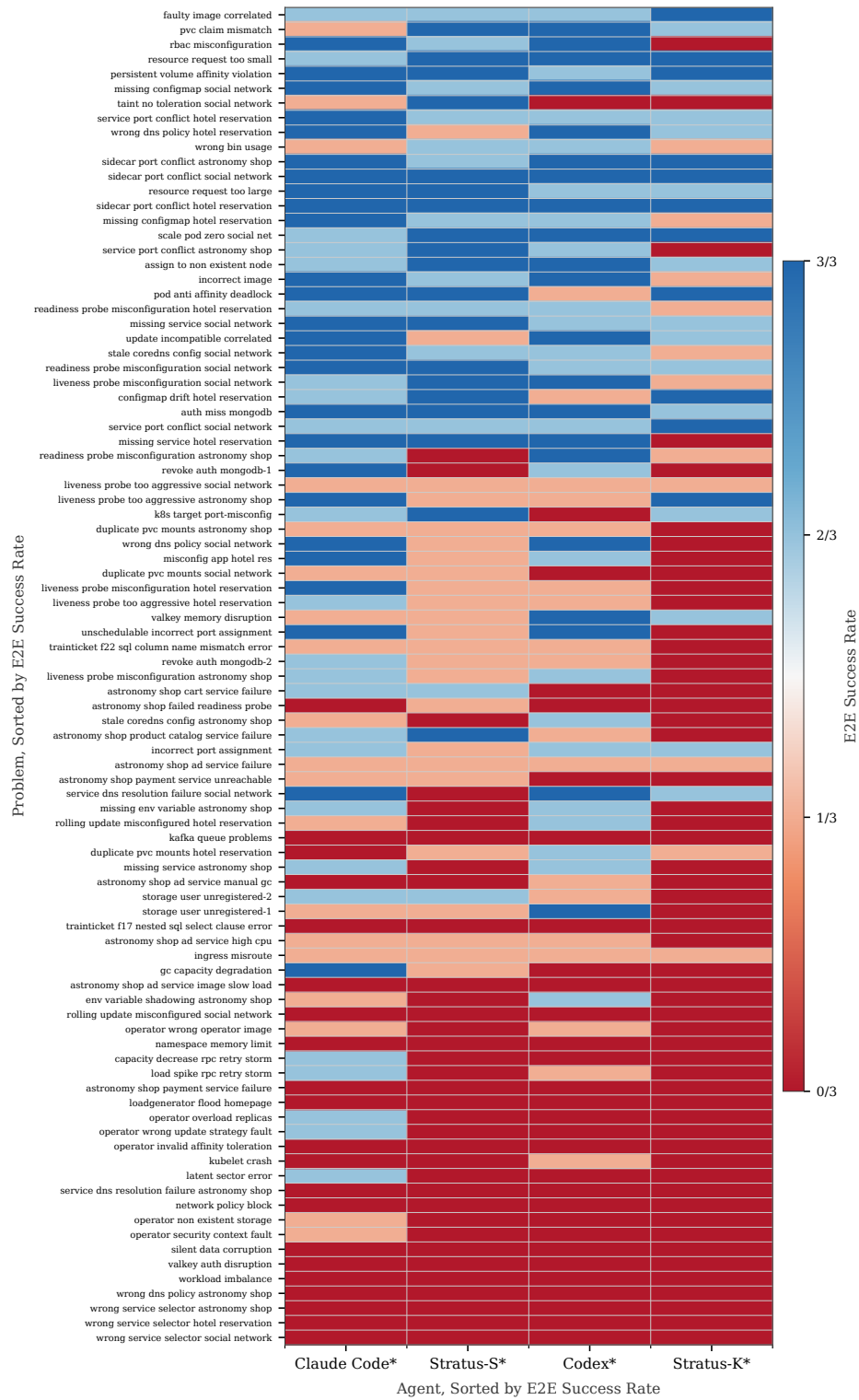


图 8: 噪声条件下的逐问题端到端成功率。问题集与图 7 相同; 智能体列按端到端成功率排序: Claude Code*、Stratus-S*、Codex*、Stratus-K* (带 * 表示噪声条件)。颜色刻度从 0/3 到 3/3 表示 E2E 成功率。